

LAPORAN PROJEK AKHIR



Mata Kuliah: Algoritma dan Pemrograman

Kelompok A – Statistika A

Penyusun:

Sekar Sandy Setiawan	(K1D025004)
Keyko Anastasia Molshac	(K1D025009)
Fatmawaty Dwi Yunianti	(K1D025013)
Julius Raymond	(K1D025032)
Ilham Luqmanulhakim	(K1D025038)

KEMENTRIAN PENDIDIKAN TINGGI, SAINS, DAN TEKNOLOGI
UNIVERSITAS JENDRAL SOEDIRMAN
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
PROGRAM STUDI STATISTIKA
PURWOKERTO
2026

KATA PENGANTAR

Puji syukur kami panjatkan kepada Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan projek mata kuliah Algoritma dan Pemrograman ini dapat diselesaikan dengan baik.

Laporan ini dibuat untuk memenuhi tugas projek akhir mata kuliah Algoritma dan Pemrograman Kelas A. Dalam projek ini kami membuat beberapa program menggunakan bahasa Python dan R berdasarkan studi kasus yang telah diberikan.

Kami menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat kami harapkan demi perbaikan laporan di masa mendatang.

Akhir kata, kami ucapkan terima kasih kepada Lutfiah Maharani Siniwi sebagai dosen pengampu mata kuliah Algoritma dan Pemrograman, serta semua pihak yang telah berperan serta dalam penyusunan laporan ini dari awal sampai akhir.

Purwokerto, Juni 2026

Penulis

DAFTAR ISI

BAB IPENDAHULUAN	3
1.1. Latar Belakang	3
1.2. Tujuan Project.....	4
1.3 Ruang Lingkup Project	4
BAB IIANALISIS MASALAH	6
2.1 Uraian Keenam Soal	6
2.1.1 Program 1	6
2.1.2 Program 2	6
2.1.3 Program 3	6
2.1.4 Program 4	6
2.1.5 Program 5	7
2.1.6 Program 6	7
2.2. Tabel Input – Proses – Output.....	7
2.2.1 Program 1 – Menghitung Jumlah Kata dan Kalimat	7
2.2.2 Program 2 – Menampilkan Kalimat Bertingkat	8
2.2.3 Program 3 – Menentukan Akar Persamaan Kuadrat.....	8
2.2.4 Program 4 – Menentukan Tanggal Lahir ASN	9
2.2.5 Program 5 – Menentukan Cluster Data	9
2.2.6 Program 6 – Interval Kepercayaan Proporsi	10
2.3. Batasan Masalah	10
2.3.1 Program 1 – Menghitung Jumlah Kata dan Jumlah Kalimat.....	10
2.3.2 Program 2 – Menampilkan Kalimat	10
2.3.3 Program 3 – Menghitung Akar Persamaan Kuadrat.....	11
2.3.4 Program 4 – Menentukan Tanggal Lahir ASN dari NIP	11
2.3.5 Program 5 – Menentukan Cluster Titik Data	11
2.3.6 Program 6 – Menghitung Interval Kepercayaan Proporsi	11
2.4. Kondisi Khusus yang Harus Ditangani Program	11
2.4.1. Program 1	11
2.4.2 Program 2	11

2.4.3 Program 3	12
2.4.4 Program 4	12
2.4.5 Program 5	12
2.4.6 Program 6	12
BAB III PSEUDOCODE dan FLOWCHART	12
3.1 Pseudocode	13
3.1.1 Program 1	13
3.1.2 Program 2	13
3.1.3 Program 3	14
3.1.4 Program 4	15
3.1.5 Program 5	17
3.1.6 Program 6	18
3.2 Flowchart	19
3.2.1 Program 1	19
3.2.2 Program 2	20
3.2.3 Program 3	20
3.2.4 Program 4	21
3.2.5 Program 5	22
3.2.6 Program 6	23
BAB IV	24
4.1 Code Python	25
4.1.1 Program 1	25
4.1.2 Program 2	26
4.1.3 Program 3	26
4.1.4 Program 4	27
4.1.5 Program 5	30
4.1.6 Program 6	33
4.2 Code R	36
4.2.1 Program 1	36
4.2.2 Program 2	38
4.2.3 Program 3	38
4.2.4 Program 4	40

4.2.5 Program 5	45
4.2.6 Program 6	48
BAB V	50
5.1 Tabel Penjelasan Code R.....	51
5.1.1 Program 1	51
5.1.2 Program 2	51
5.1.3 Program 3	52
5.1.4 Program 4	53
5.1.5 Program 5	56
5.1.6 Program 6	58
5.2 Tabel Penjelasan Code Python	59
5.2.1 Program 1	59
5.2.2 Program 2	60
5.2.3 Program 3	60
5.2.4 Program 4	61
5.2.5 Program 5	64
5.2.6 Program 6	66
BAB VI PENGUJIAN PROGRAM	68
6.1 Tabel Pengujian	69
6.1.1 Program 1	69
6.1.2 Program 2	69
6.1.3 Program 3	70
6.1.4 Program 4	70
6.1.5 Program 5	70
6.1.6 Program 6	71
6.2 Screenshot Hasil Running.....	71
6.2.1 Python	71
6.2.2 R.....	73
6.3 Interpretasi Output.....	75
6.3.1 Program 1	75
6.3.2 Program 2	75
6.3.3 Program 3	75

6.3.4 Program 4	75
6.3.5 Program 5	76
6.3.6 Program 6	76
BAB VIIPENUTUP	76
7.1 Kesimpulan	77
7.2 Kendala yang Dihadapi	77
7.3 Saran Pengembangan Program	77
LAMPIRAN	78

BAB I

PENDAHULUAN

1.1. Latar Belakang

Algoritma merupakan serangkaian langkah yang disusun secara sistematis untuk menyelesaikan suatu masalah dan menjadi dasar dalam pembuatan program komputer. Dalam proses pemrograman, algoritma digunakan sebagai pedoman untuk merancang solusi yang kemudian diterjemahkan ke dalam bahasa pemrograman agar dapat dijalankan oleh komputer. Oleh karena itu, pemahaman mengenai algoritma dan pemrograman sangat penting untuk mengembangkan kemampuan berpikir logis, analitis, dan terstruktur.

Bahasa pemrograman digunakan untuk menuliskan instruksi yang akan dieksekusi oleh komputer. Saat ini terdapat berbagai bahasa pemrograman yang dapat digunakan, salah satunya Python dan R yang termasuk ke dalam bahasa pemrograman tingkat tinggi sehingga lebih mudah dipahami dan digunakan dalam proses pembelajaran. Penguasaan algoritma dan bahasa pemrograman menjadi salah satu keterampilan penting dalam bidang teknologi informasi karena dapat membantu dalam menyelesaikan berbagai masalah secara efektif, efisien, dan terstruktur.

1.2. Tujuan Project

1. Memahami dan menerapkan konsep dasar algoritma serta pemrograman dalam menyelesaikan berbagai permasalahan komputasional.
2. Melatih kemampuan menganalisis masalah dengan menentukan input, proses, output, dan batasan masalah yang sesuai.
3. Mengembangkan kemampuan menyusun algoritma dalam bentuk pseudocode dan flowchart secara sistematis.
4. Mengimplementasikan algoritma ke dalam bahasa pemrograman Python dan R.
5. Melatih kemampuan berpikir logis, analitis, dan terstruktur dalam merancang solusi pemrograman.

6. Menguji dan mengevaluasi program yang dibuat untuk memastikan hasil yang diperoleh sesuai dengan kebutuhan dan tujuan program.
7. Meningkatkan kemampuan dokumentasi serta penyajian hasil pengembangan program secara sistematis dan mudah dipahami.

1.3 Ruang Lingkup Project

Ruang lingkup project Algoritma dan Pemrograman ini mencakup perancangan algoritma, pembuatan pseudocode dan flowchart, serta implementasi program menggunakan bahasa pemrograman Python dan R untuk menyelesaikan beberapa permasalahan komputasional. Adapun cakupan project meliputi:

1. Analisis teks untuk menghitung jumlah kata dan jumlah kalimat dari suatu teks.
2. Pembuatan dan manipulasi objek string sesuai format yang ditentukan.
3. Perhitungan akar-akar persamaan kuadrat berdasarkan nilai koefisien yang diberikan.
4. Pengolahan data Nomor Induk Pegawai (NIP) ASN untuk menentukan tanggal lahir berdasarkan informasi yang terkandung dalam NIP.
5. Klasifikasi titik pada ruang tiga dimensi ke dalam cluster tertentu berdasarkan jarak terdekat dari pusat cluster.
6. Perhitungan interval konfidensi proporsi berdasarkan proporsi sampel dan tingkat kepercayaan yang diberikan.
7. Penyusunan algoritma dalam bentuk pseudocode dan flowchart untuk setiap program.
8. Implementasi program menggunakan bahasa pemrograman Python dan R dengan logika yang setara.
9. Pengujian program menggunakan beberapa skenario uji serta dokumentasi hasil pengujian untuk memastikan program berjalan sesuai dengan kebutuhan.

Ruang lingkup project ini dibatasi pada penyelesaian enam studi kasus yang telah ditentukan dalam tugas, tanpa membahas pengembangan sistem yang lebih kompleks atau penggunaan teknologi di luar materi dasar algoritma dan pemrograman.

BAB II

ANALISIS MASALAH

2.1 Uraian Keenam Soal

2.1.1 Program 1

Program ini bertujuan untuk menghitung jumlah kata dan jumlah kalimat yang terdapat pada sebuah teks. Teks yang digunakan sebagai masukan merupakan sebuah artikel singkat mengenai media social. Dalam prosesnya, program harus mampu membaca keseluruhan isi teks, kemudian mengidentifikasi kata-kata yang terdapat di dalamnya serta menentukan banyaknya kalimat berdasarkan tanda akhir kalimat yang digunakan. Hasil akhir yang ditampilkan berupa jumlah kalimat yang terdapat pada teks tersebut. Program ini bermanfaat untuk melakukan analisis sederhana terhadap suatu dokumen atau artikel.

2.1.2 Program 2

Program ini bertujuan untuk menampilkan beberapa kalimat yang telah ditentukan ke layar dengan format yang sesuai dengan contoh keluaran pada soal. Terdapat empat buah string, yaitu K1, K2, K3, dan K4, yang harus dicetak pada posisi dan urutan yang benar. Fokus utama dari program ini adalah penggunaan perintah keluaran (output) dan pengaturan format tampilan teks sehingga hasil yang muncul di layar sama dengan yang diharapkan. Program ini melatih kemampuan dasar dalam manipulasi string dan penggunaan fungsi output dalam bahasa pemrograman.

2.1.3 Program 3

Program digunakan untuk mencari akar-akar persamaan kuadrat:

$$ax^2 + bx + c = 0$$

dengan menggunakan rumus kuadrat. Pengguna diminta memasukkan nilai koefisien a , b , dan c , kemudian program akan menghitung nilai diskriminan ($b^2 - 4ac$). Berdasarkan nilai diskriminan tersebut, program dapat menentukan jenis akar yang diperoleh, yaitu dua akar real berbeda, dua akar real kembar, atau dua akar imajiner. Selanjutnya program menghitung nilai akar-akar persamaan dan menampilkannya hingga tiga angka di belakang koma. Program ini membantu pengguna dalam menyelesaikan permasalahan matematika yang berkaitan dengan persamaan kuadrat secara otomatis dan akurat.

2.1.4 Program 4

Program ini bertujuan untuk memperoleh informasi tanggal lahir seorang Aparatur Sipil Negara (ASN) berdasarkan Nomor Pokok Pegawai (NPP) yang dimiliki. Pada format NPP yang diberikan, terdapat bagian-bagian tertentu yang menyatakan tanggal, bulan, dan tahun lahir pegawai. Program harus mampu mengambil atau mengekstrak bagian-bagian tersebut dari NPP, kemudian menyusunnya kembali menjadi format tanggal lahir yang mudah dibaca. Dengan demikian, informasi tanggal lahir ASN dapat diperoleh secara otomatis tanpa harus melakukan pemeriksaan secara manual terhadap setiap digit NPP.

2.1.5 Program 5

Program ini bertujuan untuk menentukan suatu titik data termasuk ke dalam Cluster A, Cluster B, atau Cluster C. Masing-masing cluster memiliki beberapa anggota data yang digunakan untuk menentukan titik pusat (centroid) cluster. Setelah centroid dari setiap cluster diperoleh, program menghitung jarak antara titik data yang diberikan dengan setiap centroid menggunakan rumus jarak Euclidean. Cluster yang memiliki jarak paling kecil terhadap titik data tersebut akan dianggap sebagai cluster tempat data tersebut berada. Program ini merupakan penerapan sederhana konsep klasifikasi dan pengelompokan data (clustering) yang banyak digunakan dalam bidang data mining dan machine learning.

2.1.6 Program 6

Program ini bertujuan untuk menghitung interval kepercayaan proporsi populasi berdasarkan data proporsi sampel yang tersedia. Perhitungan dilakukan menggunakan rumus interval kepercayaan proporsi dengan tingkat kepercayaan tertentu. Program harus mampu menghitung batas bawah dan batas atas interval kepercayaan untuk dua tingkat kepercayaan yang berbeda, yaitu 90% dan 95% dengan nilai z berturut-turut 1,645; dan 1,96. Selain itu, program juga harus menampilkan persentase kesalahan atau margin of error yang dihasilkan pada masing-masing tingkat kepercayaan. Dengan adanya program ini, pengguna dapat mengetahui rentang nilai proporsi populasi yang mungkin berdasarkan data sampel yang dimiliki.

2.2. Tabel Input – Proses – Output

2.2.1 Program 1 – Menghitung Jumlah Kata dan Kalimat

Komponen	Keterangan
Input	Teks bebas yang dimasukkan oleh pengguna.
Proses	1. Membaca teks masukan yang diberikan pengguna. 2. Memisahkan kata-kata dalam teks berdasarkan spasi, 3. Menghitung jumlah kata. 4. Mengidentifikasi jumlah kalimat berdasarkan tanda akhir kalimat. 5. Menghitung jumlah kalimat.
Output	Jumlah kata dan jumlah kalimat dalam teks.

2.2.2 Program 2 – Menampilkan Kalimat Bertingkat

Komponen	Keterangan
Input	String K1, K2, K3, K4 yang telah ditentukan.
Proses	1. Menyimpan setiap kalimat ke dalam variabel. 2. Menampilkan setiap kalimat sesuai urutan dan format yang diminta. 3. Mengatur perpindahan baris saat output ditampilkan.
Output	Kalimat tampil sesuai format pada soal.

2.2.3 Program 3 – Menentukan Akar Persamaan Kuadrat

Komponen	Keterangan
Input	Nilai koefisien a, b, dan c.
Proses	1. Menghitung nilai diskriminan ($D = b^2 - 4ac$).

	- Menentukan jenis akar berdasarkan nilai diskriminan. - Menghitung akar-akar persamaan menggunakan rumus kuadrat. - Membulatkan hasil hingga tiga angka decimal.
Output	Nilai akar-akar persamaan kuadrat (real atau imajiner).

2.2.4 Program 4 – Menentukan Tanggal Lahir ASN

Komponen	Keterangan
Input	Nomor pokok pegawai (NPP).
Proses	- Membaca NPP yang dimasukkan. - Mengambil bagian digit yang menunjukkan tanggal lahir. - Mengambil bagian digit yang menunjukkan bulan lahir. - Mengambil bagian digit yang menunjukkan tahun lahir. - Menyusun informasi tanggal lahir.
Output	Tanggal lahir ASN dalam format tanggal, bulan, dan tahun.

2.2.5 Program 5 – Menentukan Cluster Data

Komponen	Keterangan
Input	Koordinat titik data (x,y).
Proses	- Menentukan centroid Cluster A,B, dan C. - Menghitung jarak Euclidean titik data ke setiap centroid. - Membandingkan ketiga jarak tersebut.

	4. Menentukan cluster dengan jarak terkecil.
Output	Nama cluster tempat titik data berada (A,B, atau C).

2.2.6 Program 6 – Interval Kepercayaan Proporsi

Komponen	Keterangan
Input	Proporsi sampel $\hat{p}$ berupa bilangan decimal antara 0 dan 1, ukuran sampel, dan Tingkat kepercayaan.
Proses	1. Menentukan nilai Z sesuai Tingkat kepercayaan. 2. Menghitung standar error proporsi. 3. Menghitung margin of error. 4. Menghitung batas bawah interval kepercayaan. 5. Menghitung batas atas interval kepercayaan.
Output	Interval kepercayaan proporsi dan presentase kesalahan (margin of error).

2.3. Batasan Masalah

2.3.1 Program 1 – Menghitung Jumlah Kata dan Jumlah Kalimat

- Teks masukan berupa satu string teks.
- Tanda titik (.) hanya digunakan sebagai penanda akhir kalimat, bukan sebagai singkatan atau decimal.
- Kata dipisahkan oleh spasi.
- Program hanya menghitung jumlah kata dan jumlah kalimat.

2.3.2 Program 2 – Menampilkan Kalimat

- Kalimat yang ditampilkan sudah ditentukan dalam program.
- Program tidak menerima masukan dari pengguna.
- Program hanya menampilkan teks sesuai format yang diberikan.

- Isi kalimat tidak diubah selama proses eksekusi.

2.3.3 Program 3 – Menghitung Akar Persamaan Kuadrat

- Persamaan berbentuk $ax^2 + bx + c = 0$.
- Nilai a tidak boleh sama dengan 0.
- Koefisien a , b , dan c berupa bilangan real.
- Perhitungan akar dilakukan menggunakan rumus kuadrat.

2.3.4 Program 4 – Menentukan Tanggal Lahir ASN dari NIP

- NIP mengikuti format resmi ASN.
- Delapan digit pertama NIP menyatakan tanggal lahir.
- NIP hanya terdiri atas angka.
- Program hanya menampilkan informasi tanggal lahir berdasarkan NIP yang diberikan.

2.3.5 Program 5 – Menentukan Cluster Titik Data

- Titik data dan pusat cluster memiliki tiga koordinat (x, y, z) .
- Semua koordinat berupa nilai numerik.
- Penentuan cluster dilakukan berdasarkan jarak Euclidean.

2.3.6 Program 6 – Menghitung Interval Kepercayaan Proporsi

- Nilai proporsi sampel ($\hat{p}$) berada pada rentang 0 sampai 1.
- Ukuran sampel (n) lebih besar dari 0.
- Tingkat signifikansi yang digunakan hanya 0,05 dan 0,10.
- Perhitungan dilakukan untuk satu proporsi populasi berdasarkan data sampel yang diberikan.

2.4. Kondisi Khusus yang Harus Ditangani Program

2.4.1. Program 1

- Teks kosong sehingga jumlah kata dan jumlah kalimat bernilai 0.
- Teks tidak memiliki tanda titik sehingga jumlah kalimat bernilai 0.
- Teks hanya terdiri dari satu kalimat.
- Terdapat spasi berlebih dalam teks.

2.4.2 Program 2

- Kalimat mengandung tanda petik tunggal.
- Kalimat mengandung tanda petik ganda.
- Kalimat mengandung kombinasi tanda petik tunggal dan petik ganda.
- Salah satu string kosong.

2.4.3 Program 3

- Nilai diskriminan lebih besar dari 0 sehingga diperoleh dua akar real berbeda.
- Nilai diskriminan sama dengan 0 sehingga diperoleh akar real kembar.
- Nilai diskriminan kurang dari 0 sehingga diperoleh akar imajiner.
- Nilai koefisien (a) sama dengan 0 sehingga persamaan bukan merupakan persamaan kuadrat.

2.4.4 Program 4

- Nilai bulan hasil ekstraksi tidak berada pada rentang 1–12.
- Panjang NIP tidak sesuai dengan format yang ditentukan.
- NIP mengandung karakter selain angka.
- Tanggal lahir yang diperoleh tidak valid.

2.4.5 Program 5

- Titik data tepat berada pada pusat suatu cluster.
- Jarak titik data ke dua cluster bernilai sama.
- Jarak titik data ke ketiga cluster bernilai sama.
- Koordinat titik data bernilai negatif atau desimal.

2.4.6 Program 6

- Nilai proporsi sampel kurang dari 0 atau lebih dari 1.
- Nilai alpha selain 0,05 dan 0,10.
- Ukuran sampel sangat kecil.
- Nilai proporsi sampel mendekati 0 atau 1.

BAB III

PSEUDOCODE dan FLOWCHART

3.1 Pseudocode

3.1.1 Program 1

START

Input teks

Jumlah_kalimat $\leftarrow$ 0

For setiap karakter In teks Do

If karakter = “.” Then

Jumlah_kalimat $\leftarrow$ Jumlah_kalimat + 1

End if

End for

Daftar_kata $\leftarrow$ split(teks, “”)

Jumlah_kata $\leftarrow$ length(Daftar_kata)

Output “Teks tersebut memuat”, jumlah_kalimat, “kalimat”

Output “Teks tersebut memuat”, jumlah_kata, “kata”

END

3.1.2 Program 2

START

K1 $\leftarrow$ "Saya tak 'kan menyerah"

K2 $\leftarrow$ "Ia berkata, 'Aku Menyayangimu'"

K3 $\leftarrow$ "Coba jelaskan pengertian cross-validation dalam Machine Learning!"

K4 $\leftarrow$ "Surat keputusan itu bernomor 62/UN.34/19/2023."

Output K1

Output K2

Output K3

Output K4

END

3.1.3 Program 3

START

Input a

Input b

Input c

$D \leftarrow b^2 - 4 \times a \times c$

if $D > 0$ then

$x1 \leftarrow (-b + \sqrt{D}) / (2 \times a)$

$x2 \leftarrow (-b - \sqrt{D}) / (2 \times a)$

Output x1 dan x2

else if $D = 0$ then

$x \leftarrow -b / (2 \times a)$

Output x

else

Output "Persamaan hanya memiliki akar imajiner"

end if

END

3.1.4 Program 4

START

Input NIP

tahun $\leftarrow$ digit 1 sampai 4

bulan $\leftarrow$ digit 5 sampai 6

tanggal $\leftarrow$ digit 7 sampai 8

#Menentukan tahun kabisat

IF (tahun MOD 4 == 0 AND tahun MOD 100 $\neq$ 0) OR (tahun MOD 400 == 0) THEN

kabisat $\leftarrow$ TRUE

ELSE kabisat $\leftarrow$ FALSE

ENDIF

IF bulan $\in$ {1,3,5,7,8,10,12} THEN

maks $\leftarrow$ 31

ELSE IF bulan $\in$ {4,6,9,11} THEN

maks $\leftarrow$ 30

ELSE IF bulan == 2 AND kabisat == TRUE THEN

maks $\leftarrow$ 29

ELSE IF bulan == 2 AND kabisat == FALSE THEN

maks $\leftarrow$ 28

ELSE

maks $\leftarrow$ 0

ENDIF

IF bulan = 1 THEN

nama_bulan ← "Januari"

ELSE IF bulan = 2 THEN

nama_bulan ← "Februari"

ELSE IF bulan = 3 THEN

nama_bulan ← "Maret"

ELSE IF bulan = 4 THEN

nama_bulan ← "April"

ELSE IF bulan = 5 THEN

nama_bulan ← "Mei"

ELSE IF bulan = 6 THEN

nama_bulan ← "Juni"

ELSE IF bulan = 7 THEN

nama_bulan ← "Juli"

ELSE IF bulan = 8 THEN

nama_bulan ← "Agustus"

ELSE IF bulan = 9 THEN

nama_bulan ← "September"

ELSE IF bulan = 10 THEN

nama_bulan ← "Oktober"

ELSE IF bulan = 11 THEN

nama_bulan ← "November"

ELSE IF bulan = 12 THEN

nama_bulan ← "Desember"

ELSE

nama_bulan $\leftarrow$ "Tidak diketahui"

END IF

Output ("Tanggal lahir:", tanggal, nama_bulan, tahun)

END

3.1.5 Program 5

START

function jarak (P, Q)

jarak $\leftarrow \sqrt{(P1-Q1)^2 + (P2-Q2)^2 + (P3-Q3)^2}$

return jarak

end function

A $\leftarrow$ (2,1,3)

B $\leftarrow$ (1,-4,6)

C $\leftarrow$ (-2,3,-2)

input x1

input x2

input x3

U $\leftarrow$ (x1,x2,x3)

dA $\leftarrow$ Jarak(U,A)

dB $\leftarrow$ Jarak(U,B)

dC $\leftarrow$ Jarak(U,C)

if dA < dB and dA < dC then

cluster $\leftarrow$ "A"

else if dB < dA and dB < dC then

```

    cluster ← "B"

else if dC < dA && dC < dB then

    cluster ← "C"

else if dA == dB && dA < dC then

    cluster ← "Perbatasan cluster A dan B"

else if dA == dC && dA < dB then

    cluster ← "Perbatasan cluster A dan C"

else if dB == dC && dB < dA then

    cluster ← "Perbatasan cluster B dan C"

else

    cluster ← "Perbatasan cluster A, B, dan C"

end if

output "Titik termasuk cluster ", cluster

END

```

3.1.6 Program 6

START

```

function interval_proporsi(phat, n, alpha)

    if phat < 0 or phat > 1 then

        output "Error: proporsi harus antara 0 dan 1"

        return

    end if

    if alpha = 0.10 then

        z ← 1.645

    else if alpha = 0.05 then

```

```

        z ← 1.96
    else
        output "Alpha harus 0.10 atau 0.05"
        return
    end if

    margin ← z × √((phat × (1-phat))/n)

    bawah ← phat – margin
    atas ← phat + margin

    output "Interval Konfidensi"
    output "(", bawah, ",", atas, ")"
end function

input phat
input n
input alpha

call interval_proporsi(phat, n, alpha)

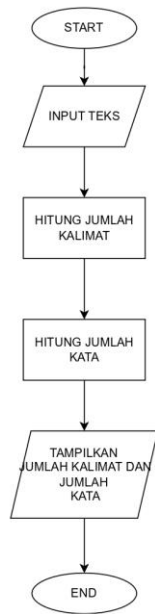
END

```

3.2 Flowchart

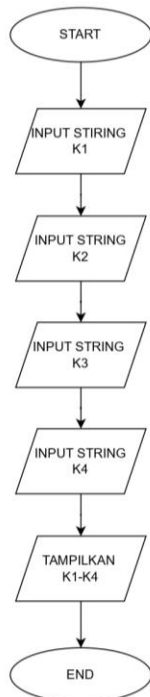
3.2.1 Program 1

PROGRAM 1

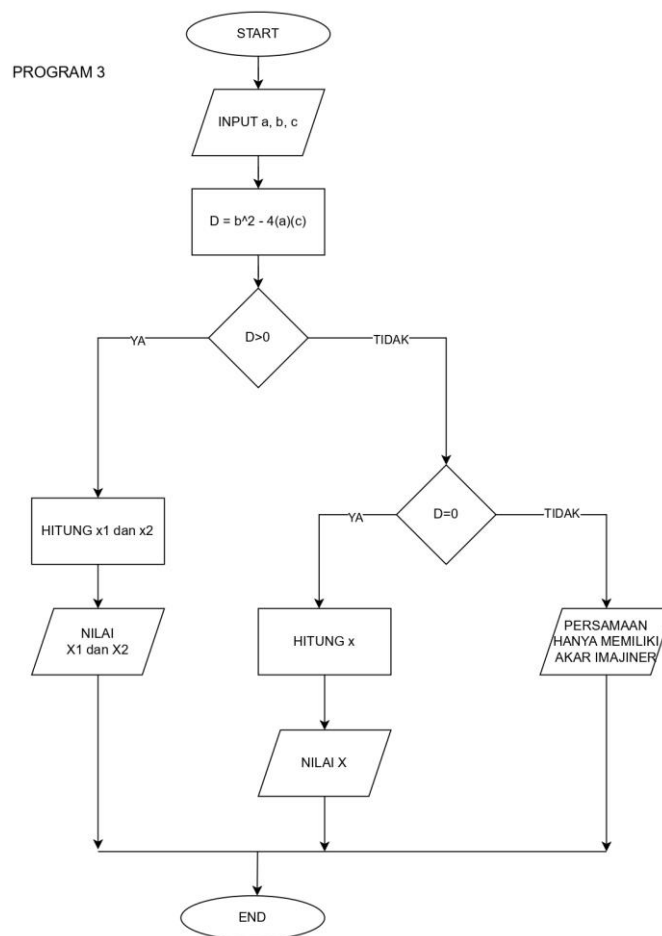


3.2.2 Program 2

PROGRAM 2

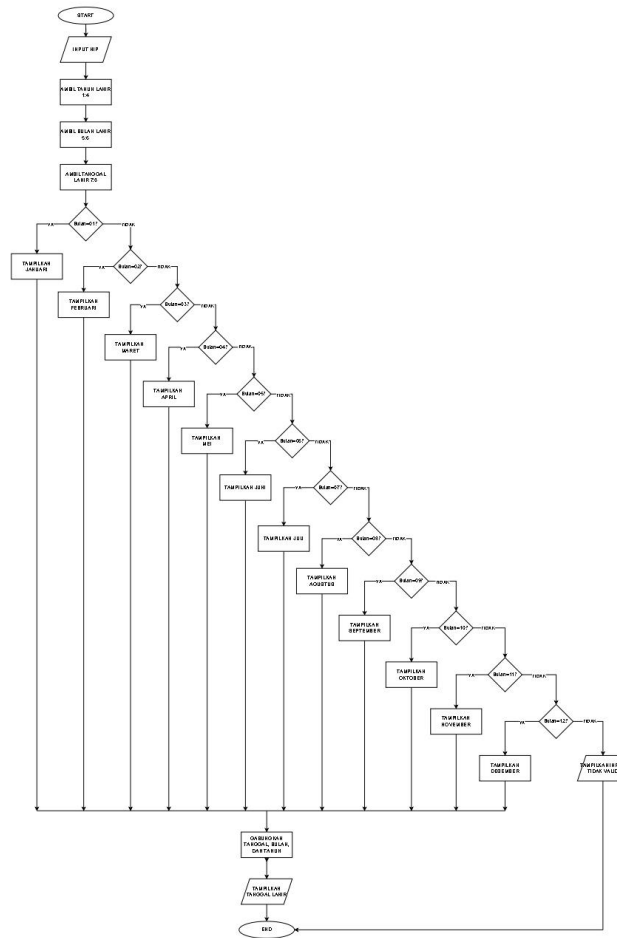


3.2.3 Program 3

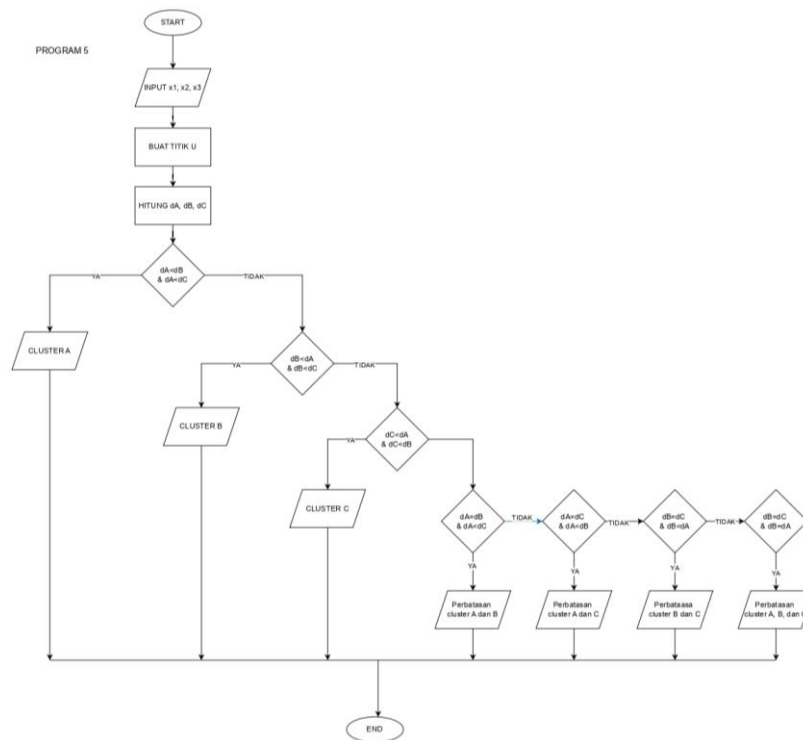


3.2.4 Program 4

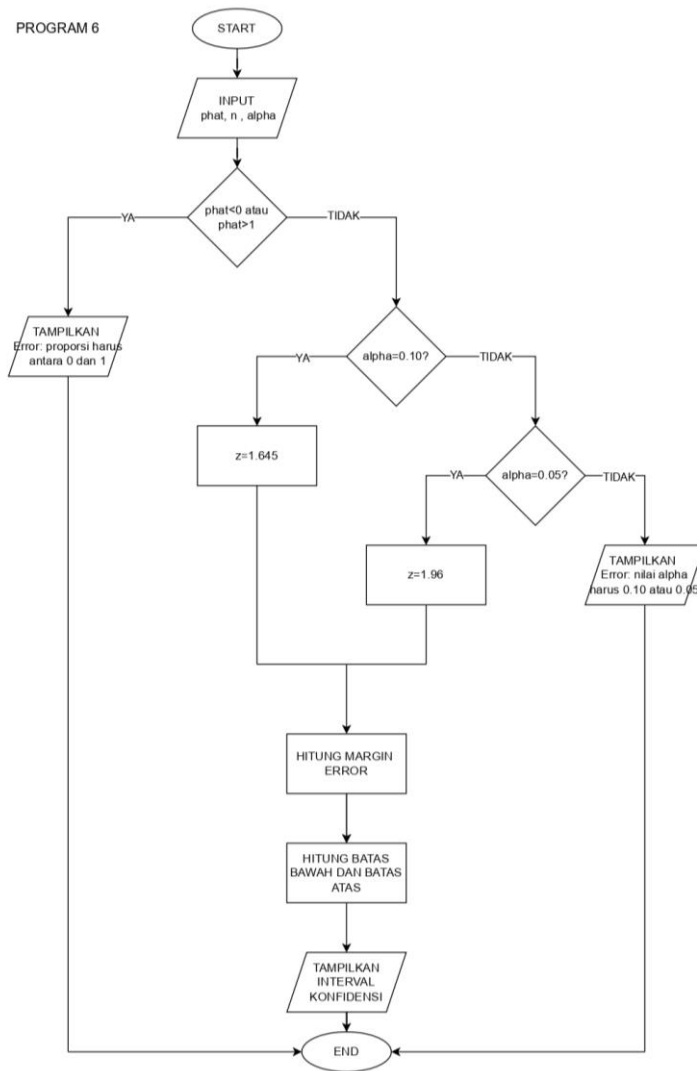
PROGRAM 4



3.2.5 Program 5



3.2.6 Program 6



BAB IV

SOURCE CODE PYHON AND R

4.1 Code Python

4.1.1 Program 1

```
#Program 1 Menghitung banyaknya kata dan banyaknya kalimat
#Keadaan 1
teks= "Kehidupan anak statistika tidak jauh dari data, angka, dan berbagai tugas analisis.
Setiap hari mereka belajar mengolah data untuk menemukan informasi yang dapat
digunakan dalam pengambilan keputusan. Meskipun sering berhadapan dengan
perhitungan yang rumit, anak statistika tetap menikmati proses belajar karena ilmu yang
dipelajari sangat bermanfaat di berbagai bidang."

# Menghitung banyaknya kalimat dan banyaknya kata
banyaknya_kalimat = teks.count('.')
banyaknya_kata= len(teks.split())
print('Banyaknya Kalimat adalah',banyaknya_kalimat)
print('Banyaknya Kata adalah',banyaknya_kata)

#Keadaan 2
teks= """Olahraga merupakan aktivitas fisik yang sangat penting bagi kesehatan tubuh
manusia. Dengan berolahraga secara teratur tubuh kita akan menjadi lebih bugar dan
kuat. Banyak jenis olahraga yang bisa dipilih sesuai dengan minat dan kemampuan
masing masing. Sepak bola basket renang dan lari adalah beberapa olahraga yang
populer di Indonesia. Olahraga juga terbukti dapat mengurangi stres dan meningkatkan
kesehatan mental seseorang. Pemerintah terus mendorong masyarakat untuk aktif
berolahraga melalui berbagai program dan fasilitas umum. Mulailah berolahraga dari
sekarang karena lebih baik mencegah daripada mengobati penyakit."""

# Menghitung banyaknya kalimat dan banyaknya kata
banyaknya_kalimat = teks.count('.')
banyaknya_kata= len(teks.split())
print('Banyaknya Kalimat adalah',banyaknya_kalimat)
print('Banyaknya Kata adalah',banyaknya_kata)

#Keadaan 3
teks= """Indonesia memiliki kekayaan kuliner tradisional yang sangat beragam dan
lezat. . Setiap daerah di Indonesia memiliki makanan khas yang mencerminkan budaya
setempat. Rendang dari Sumatera Barat diakui sebagai salah satu makanan terlezat di
dunia. Soto gudeg nasi goreng dan gado gado juga merupakan makanan yang sangat
terkenal. Makanan tradisional Indonesia menggunakan rempah rempah alami yang kaya
manfaat bagi kesehatan. Kita harus melestarikan kuliner tradisional agar tidak punah
```

tergerus oleh makanan modern. Memperkenalkan makanan tradisional kepada generasi muda adalah tanggung jawab kita bersama. Dengan mencintai kuliner lokal kita turut menjaga warisan budaya bangsa Indonesia."""

```
# Menghitung banyaknya kalimat dan banyaknya kata
banyaknya_kalimat = teks.count('.')
banyaknya_kata= len(teks.split())
print('Banyaknya Kalimat adalah',banyaknya_kalimat)
print('Banyaknya Kata adalah',banyaknya_kata)
```

4.1.2 Program 2

```
#Program 2 Menampilkan Kalimat
K1= "Saya tak 'kan menyerah"
K2= "Ia berkata,\"Aku Menyayangimu\"."
K3= "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."

print(K1)
print(K2)
print(K3)
print(K4)
```

4.1.3 Program 3

```
#Program 3 Mencari Akar-Akar Persamaan Kuadrat  $ax^2 + bx + c = 0$ 
#Kondisi 1
import math
a = 1
b = 4
c = 4
d = ( b**2 - 4*a*c)
if d>0:
    x1 = (-b + math.sqrt(d))/(2*a)
    x2 = (-b - math.sqrt(d))/(2*a)
    print("Akar-akarnya adalah",x1,"dan",x2)
elif d == 0:
    x = -b/(2*a)
    print("Akar-akarnya adalah",x)
else:
    print("Persamaan hanya memiliki akar imajiner")

#Kondisi 2
import math
```

```

a = 2
b = 3
c = 6
d = ( b**2 - 4*a*c)
if d>0:
    x1 = (-b + math.sqrt(d))/(2*a)
    x2 = (-b - math.sqrt(d))/(2*a)
    print("Akar-akarnya adalah",x1,"dan",x2)
elif d == 0:
    x = -b/(2*a)
    print("Akar-akarnya adalah",x)
else:
    print("Persamaan hanya memiliki akar imajiner")

```

#Kondisi 3

```

import math
a = 1
b = -5
c = 6
d = ( b**2 - 4*a*c)
if d>0:
    x1 = (-b + math.sqrt(d))/(2*a)
    x2 = (-b - math.sqrt(d))/(2*a)
    print("Akar-akarnya adalah",x1,"dan",x2)
elif d == 0:
    x = -b/(2*a)
    print("Akar-akarnya adalah",x)
else:
    print("Persamaan hanya memiliki akar imajiner")

```

4.1.4 Program 4

```
# Program 4 Menampilkan Tanggal Lahir ASN dari NIP
#Kondisi 1
nip = str(199904212019031010)
tahun = nip[0:4]
bulan = int(nip[4:6])
tanggal = int(nip[6:8])
if tanggal < 1 or tanggal > 31:
    print("Tanggal lahir tidak valid")
if bulan == 1:
    nama_bulan = "Januari"
elif bulan == 2:
    nama_bulan = "Februari"
elif bulan == 3:
    nama_bulan = "Maret"
elif bulan == 4:
    nama_bulan = "April"
elif bulan == 5:
    nama_bulan = "Mei"
elif bulan == 6:
    nama_bulan = "Juni"
elif bulan == 7:
    nama_bulan = "Juli"
elif bulan == 8:
    nama_bulan = "Agustus"
elif bulan == 9:
    nama_bulan = "September"
elif bulan == 10:
    nama_bulan = "Oktober"
elif bulan == 11:
    nama_bulan = "November"
elif bulan == 12:
    nama_bulan = "Desember"
else:
    nama_bulan = "Bulan tidak valid"
print(f"Tanggal lahir: {tanggal} {nama_bulan} {tahun}.")

#Kondisi 2
nip = str(199609162019031008)
tahun = nip[0:4]
bulan = int(nip[4:6])
tanggal = int(nip[6:8])
if tanggal < 1 or tanggal > 31:
```

```

    print("Tanggal lahir tidak valid")
if bulan == 1:
    nama_bulan = "Januari"
elif bulan == 2:
    nama_bulan = "Februari"
elif bulan == 3:
    nama_bulan = "Maret"
elif bulan == 4:
    nama_bulan = "April"
elif bulan == 5:
    nama_bulan = "Mei"
elif bulan == 6:
    nama_bulan = "Juni"
elif bulan == 7:
    nama_bulan = "Juli"
elif bulan == 8:
    nama_bulan = "Agustus"
elif bulan == 9:
    nama_bulan = "September"
elif bulan == 10:
    nama_bulan = "Oktober"
elif bulan == 11:
    nama_bulan = "November"
elif bulan == 12:
    nama_bulan = "Desember"
else:
    nama_bulan = "Bulan tidak valid"
print(f"Tanggal lahir: {tanggal} {nama_bulan} {tahun}.")

```

Program 4 Menampilkan Tanggal Lahir ASN dari NIP

#Kondisi 3

```

nip = str(199513152019031006)
tahun = nip[0:4]
bulan = int(nip[4:6])
tanggal = int(nip[6:8])
if tanggal < 1 or tanggal > 31:
    print("Tanggal lahir tidak valid")
if bulan == 1:
    nama_bulan = "Januari"
elif bulan == 2:
    nama_bulan = "Februari"
elif bulan == 3:
    nama_bulan = "Maret"
elif bulan == 4:

```



```

        nama_bulan = "April"
elif bulan == 5:
    nama_bulan = "Mei"
elif bulan == 6:
    nama_bulan = "Juni"
elif bulan == 7:
    nama_bulan = "Juli"
elif bulan == 8:
    nama_bulan = "Agustus"
elif bulan == 9:
    nama_bulan = "September"
elif bulan == 10:
    nama_bulan = "Oktober"
elif bulan == 11:
    nama_bulan = "November"
elif bulan == 12:
    nama_bulan = "Desember"
else:
    nama_bulan = "Bulan tidak valid"
print(f'Tanggal lahir: {tanggal} {nama_bulan} {tahun}.")

```

4.1.5 Program 5

#PROGRAM 5

```

##Keadaan 1
"""

```

```

import math

```

```

#Titik pusat cluster

```

```

A=(2,1,3)

```

```

B=(1,-4,6)

```

```

C=(-2,3,-2)

```

```

#input titik U

```

```

x1=1

```

```

x2=-4

```

```

x3=6

```

```

U=(x1,x2,x3)

```

```

#Rumus jarak:

```

```

def jarak(p, q):

```

```

    hasil_jarak=math.sqrt((p[0]-q[0])**2 + (p[1]-q[1])**2 + (p[2]-q[2])**2)

```

```
return hasil_jarak
```

```
#Menghitung jarak ke pusat cluster
```

```
dA = jarak(U,A)
```

```
dB = jarak(U,B)
```

```
dC = jarak(U,C)
```

```
#Menentukan cluster terdekat
```

```
if dA<dB and dA<dC:
```

```
    cluster="cluster A"
```

```
elif dB<dA and dB<dC:
```

```
    cluster="cluster B"
```

```
elif dC<dA and dC< dB:
```

```
    cluster = "cluster C"
```

```
elif dA==dB and dA<dC:
```

```
    cluster="Perbatasan cluster A dan B"
```

```
elif dA==dC and dA<dB:
```

```
    cluster="Perbatasan cluster A dan C"
```

```
elif dB==dC and dB<dA:
```

```
    cluster="Perbatasan cluster B dan C"
```

```
else:
```

```
    cluster="Perbatasan cluster A, B, dan C"
```

```
#Tampilkan hasil
```

```
print(f"Koordinat titik U : {U}")
```

```
print(f"Jarak U ke A : {dA:.4f}")
```

```
print(f"Jarak U ke B : {dB:.4f}")
```

```
print(f"Jarak U ke C : {dC:.4f}")
```

```
print(f"Titik tergolong {cluster}")
```

```
""""##Keadaan 2""""
```

```
import math
```

```
#Titik pusat cluster
```

```
A=(2,1,3)
```

```
B=(1,-4,6)
```

```
C=(-2,3,-2)
```

```
#input titik U
```

```
x1=4
```

```
x2=2
```

```
x3=3
```

```
U=(x1,x2,x3)
```

```

#Rumus jarak:
def jarak(p, q):
    hasil_jarak=math.sqrt((p[0]-q[0])**2 + (p[1]-q[1])**2 + (p[2]-q[2])**2)
    return hasil_jarak

#Menghitung jarak ke pusat cluster
dA = jarak(U,A)
dB = jarak(U,B)
dC = jarak(U,C)

#Menentukan cluster terdekat
if dA<dB and dA<dC:
    cluster="cluster A"
elif dB<dA and dB<dC:
    cluster="cluster B"
elif dC<dA and dC< dB:
    cluster = "cluster C"
elif dA==dB and dA<dC:
    cluster="Perbatasan cluster A dan B"
elif dA==dC and dA<dB:
    cluster="Perbatasan cluster A dan C"
elif dB==dC and dB<dA:
    cluster="Perbatasan cluster B dan C"
else:
    cluster="Perbatasan cluster A, B, dan C"

#Tampilkan hasil
print(f"Koordinat titik U : {U}")
print(f"Jarak U ke A : {dA:.4f}")
print(f"Jarak U ke B : {dB:.4f}")
print(f"Jarak U ke C : {dC:.4f}")
print(f"Titik tergolong {cluster}")

"""##Keadaan 3

"""

import math

#Titik pusat cluster
A=(2,1,3)
B=(1,-4,6)
C=(-2,3,-2)

```

```

#input titik U
x1=1.5
x2=-1.5
x3=4.5
U=(x1,x2,x3)

#Rumus jarak:
def jarak(p, q):
    hasil_jarak=math.sqrt((p[0]-q[0])**2 + (p[1]-q[1])**2 + (p[2]-q[2])**2)
    return hasil_jarak

#Menghitung jarak ke pusat cluster
dA = jarak(U,A)
dB = jarak(U,B)
dC = jarak(U,C)

#Menentukan cluster terdekat
if dA<dB and dA<dC:
    cluster="cluster A"
elif dB<dA and dB<dC:
    cluster="cluster B"
elif dC<dA and dC< dB:
    cluster = "cluster C"
elif dA==dB and dA<dC:
    cluster="Perbatasan cluster A dan B"
elif dA==dC and dA<dB:
    cluster="Perbatasan cluster A dan C"
elif dB==dC and dB<dA:
    cluster="Perbatasan cluster B dan C"
else:
    cluster="Perbatasan cluster A, B, dan C"

#Tampilkan hasil
print(f"Koordinat titik U : {U}")
print(f"Jarak U ke A : {dA:.4f}")
print(f"Jarak U ke B : {dB:.4f}")
print(f"Jarak U ke C : {dC:.4f}")
print(f"Titik tergolong {cluster}")

```

4.1.6 Program 6

```
#PROGRAM 6
```

```
##Keadaan 1
```

```
"""
```

```
import math
```

```
# Fungsi menghitung interval konfidensi proporsi
```

```
def interval_konfidensi(p_hat, alpha, n):
```

```
    # Validasi proporsi
```

```
    if p_hat < 0 or p_hat > 1:
```

```
        print("Error: Proporsi harus antara 0 dan 1.")
```

```
        return
```

```
    # Menentukan nilai z
```

```
    if alpha == 0.10:
```

```
        z = 1.645
```

```
    elif alpha == 0.05:
```

```
        z = 1.96
```

```
    else:
```

```
        print("Error: Nilai alpha harus 0.10 (10%) atau 0.05 (5%).")
```

```
        return
```

```
    # Menghitung margin of error
```

```
    margin = z * math.sqrt((p_hat * (1 - p_hat)) / n)
```

```
    # Menghitung batas interval
```

```
    batas_bawah = p_hat - margin
```

```
    batas_atas = p_hat + margin
```

```
    # Menampilkan hasil
```

```
    print(f"Interval konfidensi      : ({batas_bawah:.4f}, {batas_atas:.4f})")
```

```
    print(f"Kesimpulan: {batas_bawah:.4f} < p < {batas_atas:.4f}")
```

```
# Memanggil fungsi
```

```
interval_konfidensi(0.3, 0.05, 15)
```

```
"""##Keadaan 2"""
```

```
import math
```

```
# Fungsi menghitung interval konfidensi proporsi
```

```
def interval_konfidensi(p_hat, alpha, n):
```

```
    # Validasi proporsi
```

```

if p_hat < 0 or p_hat > 1:
    print("Error: Proporsi harus antara 0 dan 1.")
    return

# Menentukan nilai z
if alpha == 0.10:
    z = 1.645
elif alpha == 0.05:
    z = 1.96
else:
    print("Error: Nilai alpha harus 0.10 (10%) atau 0.05 (5%).")
    return

# Menghitung margin of error
margin = z * math.sqrt((p_hat * (1 - p_hat)) / n)

# Menghitung batas interval
batas_bawah = p_hat - margin
batas_atas = p_hat + margin

# Menampilkan hasil
print(f"Interval konfidensi      : ({batas_bawah:.4f}, {batas_atas:.4f})")
print(f"Kesimpulan: {batas_bawah:.4f} < p < {batas_atas:.4f}")

# Memanggil fungsi
interval_konfidensi(1.5, 0.1, 50)

"""##Keadaan 3"""

import math

# Fungsi menghitung interval konfidensi proporsi
def interval_konfidensi(p_hat, alpha, n):

    # Validasi proporsi
    if p_hat < 0 or p_hat > 1:
        print("Error: Proporsi harus antara 0 dan 1.")
        return

    # Menentukan nilai z
    if alpha == 0.10:
        z = 1.645
    elif alpha == 0.05:
        z = 1.96

```

```

else:
    print("Error: Nilai alpha harus 0.10 (10%) atau 0.05 (5%).")
    return

# Menghitung margin of error
margin = z * math.sqrt((p_hat * (1 - p_hat)) / n)

# Menghitung batas interval
batas_bawah = p_hat - margin
batas_atas = p_hat + margin

# Menampilkan hasil
print(f"Interval konfidensi      : ({batas_bawah:.4f}, {batas_atas:.4f})")
print(f"Kesimpulan: {batas_bawah:.4f} < p < {batas_atas:.4f}")

# Memanggil fungsi
interval_konfidensi(0.6, 0.6, 30)

```

4.2 Code R

4.2.1 Program 1

```

#PROGRAM 1
#Keadaan 1
# Program Menghitung Jumlah Kata dan Kalimat
# Masukkan teks
teks <- "Kehidupan anak statistika tidak jauh dari data, angka, dan berbagai tugas
analisis. Setiap hari mereka belajar mengolah data untuk menemukan informasi
yang dapat digunakan dalam pengambilan keputusan. Meskipun sering
berhadapan dengan perhitungan yang rumit, anak statistika tetap menikmati proses
belajar karena ilmu yang dipelajari sangat bermanfaat di berbagai bidang."

# Hitung jumlah kata
# Pisahkan berdasarkan satu atau lebih spasi
kata <- (strsplit(trimws(teks), "\\s+")) [[1]]
jumlah_kata<-length(kata) #Menghitung jumlah elemen dalam vector kata

# Hitung jumlah kalimat
# Pisahkan berdasarkan tanda titik
kalimat <- strsplit(teks, "\\.") [[1]] #Asumsi tanda titik hanya untuk mengakhiri
kalimat
jumlah_kalimat<-length(kalimat) #Menghitung jumlah elemen dalam vector
kalimat

```

```
# Tampilkan hasil
cat(sprintf("Jumlah kalimat : %d kalimat\n", jumlah_kalimat))
cat(sprintf("Jumlah kata   : %d kata\n", jumlah_kata))
cat(sprintf("Teks tersebut memuat %d kalimat dan %d kata.\n",
            jumlah_kalimat, jumlah_kata))
```

#Keadaan 2

Program Menghitung Jumlah Kata dan Kalimat

Masukkan teks

```
teks <- "Olahraga merupakan aktivitas fisik yang sangat penting bagi kesehatan
tubuh manusia. Dengan berolahraga secara teratur tubuh kita akan menjadi lebih
bugar dan kuat. Banyak jenis olahraga yang bisa dipilih sesuai dengan minat dan
kemampuan masing masing. Sepak bola basket renang dan lari adalah beberapa
olahraga yang populer di Indonesia. Olahraga juga terbukti dapat mengurangi stres
dan meningkatkan kesehatan mental seseorang. Pemerintah terus mendorong
masyarakat untuk aktif berolahraga melalui berbagai program dan fasilitas umum.
Mulailah berolahraga dari sekarang karena lebih baik mencegah daripada
mengobati penyakit."
```

Hitung jumlah kata

Pisahkan berdasarkan satu atau lebih spasi

```
kata <- (strsplit(trimws(teks), "\\s+")) [[1]]
```

```
jumlah_kata<-length(kata) #Menghitung jumlah elemen dalam vector kata
```

Hitung jumlah kalimat

Pisahkan berdasarkan tanda titik

```
kalimat <- strsplit(teks, "\\.") [[1]] #Asumsi tanda titik hanya untuk mengakhiri
kalimat
```

```
jumlah_kalimat<-length(kalimat) #Menghitung jumlah elemen dalam vector
kalimat
```

Tampilkan hasil

```
cat(sprintf("Jumlah kalimat : %d kalimat\n", jumlah_kalimat))
```

```
cat(sprintf("Jumlah kata   : %d kata\n", jumlah_kata))
```

```
cat(sprintf("Teks tersebut memuat %d kalimat dan %d kata.\n",
            jumlah_kalimat, jumlah_kata))
```

#Keadaan 3

Program Menghitung Jumlah Kata dan Kalimat

Masukkan teks

```
teks <- "Indonesia memiliki kekayaan kuliner tradisional yang sangat beragam dan
lezat. . Setiap daerah di Indonesia memiliki makanan khas yang mencerminkan
budaya setempat. Rendang dari Sumatera Barat diakui sebagai salah satu makanan
terlezat di dunia. Soto gudeg nasi goreng dan gado gado juga merupakan makanan
```


yang sangat terkenal. Makanan tradisional Indonesia menggunakan rempah rempah alami yang kaya manfaat bagi kesehatan. Kita harus melestarikan kuliner tradisional agar tidak punah tergerus oleh makanan modern. Memperkenalkan makanan tradisional kepada generasi muda adalah tanggung jawab kita bersama. Dengan mencintai kuliner lokal kita turut menjaga warisan budaya bangsa Indonesia."

```
# Hitung jumlah kata
# Pisahkan berdasarkan satu atau lebih spasi
kata <- (strsplit(trimws(teks), "\\s+")) [[1]]
jumlah_kata<-length(kata) #Menghitung jumlah elemen dalam vector kata

# Hitung jumlah kalimat
# Pisahkan berdasarkan tanda titik
kalimat <- strsplit(teks, "\\.") [[1]] #Asumsi tanda titik hanya untuk mengakhiri kalimat
jumlah_kalimat<-length(kalimat) #Menghitung jumlah elemen dalam vector kalimat

# Tampilkan hasil
cat(sprintf("Jumlah kalimat : %d kalimat\n", jumlah_kalimat))
cat(sprintf("Jumlah kata   : %d kata\n", jumlah_kata))
cat(sprintf("Teks tersebut memuat %d kalimat dan %d kata.\n",
            jumlah_kalimat, jumlah_kata))
```

4.2.2 Program 2

```
#PROGRAM 2
K1 <- "Saya tak 'kan menyerah."
K2 <- "Ia berkata, \"Aku menyayangiimu.\"\" #Mengandung tanda kutip ganda (") di dalam string
# Dalam R, tambahkan \" untuk menyisipkan tanda kutip ganda di dalam string
K3 <- "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
# K4 tidak ada karakter khusus yang perlu di-escape, ditulis langsung
K4 <- "Surat keputusan itu bernomor 62/UN.34/19/2023."

#Output
cat("K1:", K1, "\n")
cat("K2:", K2, "\n")
cat("K3:", K3, "\n")
cat("K4:", K4, "\n")
```

4.2.3 Program 3

```
#Projek 3
#Menghitung akar bilangan
#Keadaan 1
#Input persamaan
a<-1
b<-4
c<-4
#Input rumus D
D<-b^2-4*a*c
print(D)
#Logika if-else
if (D>0){
  x1<-(-b+sqrt(D))/(2*a)
  x2<-(-b-sqrt(D))/(2*a)
  cat(sprintf("x1 = %.3f\n", x1))
  cat(sprintf("x2 = %.3f\n", x2))
} else if (D==0){
  x <- -b/(2*a)
  cat(sprintf("x1 = x2 = %.3f\n", x))
} else {
  cat("Persamaan hanya memiliki akar imajiner\n")
}

#Keadaan 2
#Input persamaan
a<-2
b<-3
c<-6
#Input rumus D
D<-b^2-4*a*c
print(D)
#Logika if-else
if (D>0){
  x1<-(-b+sqrt(D))/(2*a)
  x2<-(-b-sqrt(D))/(2*a)
  cat(sprintf("x1 = %.3f\n", x1))
  cat(sprintf("x2 = %.3f\n", x2))
} else if (D==0){
  x <- -b/(2*a)
  cat(sprintf("x1 = x2 = %.3f\n", x))
} else {
  cat("Persamaan hanya memiliki akar imajiner\n")
}
```

```

}

#Keadaan 3
#Input persamaan
a<-1
b<- -5
c<-6
#Input rumus D
D<-b^2-4*a*c
print(D)
#Logika if-else
if (D>0){
  x1<-(-b+sqrt(D))/(2*a)
  x2<-(-b-sqrt(D))/(2*a)
  cat(sprintf("x1 = %.3f\n", x1))
  cat(sprintf("x2 = %.3f\n", x2))
} else if (D==0){
  x <- -b/(2*a)
  cat(sprintf("x1 = x2 = %.3f\n", x))
} else {
  cat("Persamaan hanya memiliki akar imajiner\n")
}

```

4.2.4 Program 4

```

#Project 4
#Keadaan 1
#Menampilkan tanggal lahir ASN dari NIP
nip <- "199904312019031010"

#Pengkategorian nip berdasarkan urutan numeric
if (nchar(nip) != 18) {
  cat("NIP tidak valid! NIP harus terdiri dari 18 angka.\n")
} else {
  tahun <- as.numeric(substr(nip, 1, 4))
  bulan <- as.numeric(substr(nip, 5, 6))
  tanggal <- as.numeric(substr(nip, 7, 8))

  # Cek tahun kabisat untuk menentukan tanggal di bulan februari
  if ((tahun %% 4 == 0 && tahun %% 100 != 0) || (tahun %% 400 == 0)) {
    kabisat <- TRUE
  } else {
    kabisat <- FALSE
  }
}

```

```
}
```

```
#Menentukan hari maksimal per bulan
if (bulan %in% c(1, 3, 5, 7, 8, 10, 12)) {
  maks <- 31
} else if (bulan %in% c(4, 6, 9, 11)) {
  maks <- 30
} else if (bulan == 2 && kabisat) {
  maks <- 29
} else if (bulan == 2 && !kabisat) {
  maks <- 28
} else {
  maks <- 0 # bulan tidak valid
}
```

```
#Pengkategorian bulan
if (bulan < 1 || bulan > 12) {
  cat("Bulan tidak valid!\n")
}
```

```
# Validasi tanggal
} else if (tanggal < 1 || tanggal > maks) {
  cat("Tanggal tidak valid! Bulan ini maksimal", maks, "hari.\n")
}
```

```
#mengganti bulan dari numeric ke character
} else {
  if (bulan == 1) {
    nama_bulan <- "Januari"
  } else if (bulan == 2) {
    nama_bulan <- "Februari"
  } else if (bulan == 3) {
    nama_bulan <- "Maret"
  } else if (bulan == 4) {
    nama_bulan <- "April"
  } else if (bulan == 5) {
    nama_bulan <- "Mei"
  } else if (bulan == 6) {
    nama_bulan <- "Juni"
  } else if (bulan == 7) {
    nama_bulan <- "Juli"
  } else if (bulan == 8) {
    nama_bulan <- "Agustus"
  } else if (bulan == 9) {
    nama_bulan <- "September"
  } else if (bulan == 10) {
    nama_bulan <- "Oktober"
  } else if (bulan == 11) {
    nama_bulan <- "November"
  } else if (bulan == 12) {
    nama_bulan <- "Desember"
  }
}
```

```

    nama_bulan <- "Oktober"
  } else if (bulan == 11) {
    nama_bulan <- "November"
  } else if (bulan == 12) {
    nama_bulan <- "Desember"
  }
  cat("Tanggal lahir:", tanggal, nama_bulan, tahun, "\n")
}
}

```

#Keadaan 2

#Menampilkan tanggal lahir ASN dari NIP

```
nip<-199609162019031008
```

#Pengkategorian nip berdasarkan urutan numeric

```

if (nchar(nip) != 18) {
  cat("NIP tidak valid! NIP harus terdiri dari 18 angka.\n")
} else {
  tahun <- as.numeric(substr(nip, 1, 4))
  bulan <- as.numeric(substr(nip, 5, 6))
  tanggal <- as.numeric(substr(nip, 7, 8))

```

Cek tahun kabisat untuk menentukan tanggal di bulan februari

```

if ((tahun %% 4 == 0 && tahun %% 100 != 0) || (tahun %% 400 == 0)) {
  kabisat <- TRUE
} else {
  kabisat <- FALSE
}

```

#Menentukan hari maksimal per bulan

```

if (bulan %in% c(1, 3, 5, 7, 8, 10, 12)) {
  maks <- 31
} else if (bulan %in% c(4, 6, 9, 11)) {
  maks <- 30
} else if (bulan == 2 && kabisat) {
  maks <- 29
} else if (bulan == 2 && !kabisat) {
  maks <- 28
} else {
  maks <- 0 # bulan tidak valid
}

```

#Pengkategorian bulan

```
if (bulan < 1 || bulan > 12) {
```

```

cat("Bulan tidak valid!\n")

# Validasi tanggal
} else if (tanggal < 1 || tanggal > maks) {
  cat("Tanggal tidak valid! Bulan ini maksimal", maks, "hari.\n")

#mengganti bulan dari numeric ke character
} else {
  if (bulan == 1) {
    nama_bulan <- "Januari"
  } else if (bulan == 2) {
    nama_bulan <- "Februari"
  } else if (bulan == 3) {
    nama_bulan <- "Maret"
  } else if (bulan == 4) {
    nama_bulan <- "April"
  } else if (bulan == 5) {
    nama_bulan <- "Mei"
  } else if (bulan == 6) {
    nama_bulan <- "Juni"
  } else if (bulan == 7) {
    nama_bulan <- "Juli"
  } else if (bulan == 8) {
    nama_bulan <- "Agustus"
  } else if (bulan == 9) {
    nama_bulan <- "September"
  } else if (bulan == 10) {
    nama_bulan <- "Oktober"
  } else if (bulan == 11) {
    nama_bulan <- "November"
  } else if (bulan == 12) {
    nama_bulan <- "Desember"
  }
  cat("Tanggal lahir:", tanggal, nama_bulan, tahun, "\n")
}
}

#Keadaan 3
#Menampilkan tanggal lahir ASN dari NIP
nip<-199513152019031006

#Pengkategorian nip berdasarkan urutan numeric
if (nchar(nip) != 18) {
  cat("NIP tidak valid! NIP harus terdiri dari 18 angka.\n")
}

```

```

} else {
  tahun  <- as.numeric(substr(nip, 1, 4))
  bulan  <- as.numeric(substr(nip, 5, 6))
  tanggal <- as.numeric(substr(nip, 7, 8))

  # Cek tahun kabisat untuk menentukan tanggal di bulan februari
  if ((tahun %% 4 == 0 && tahun %% 100 != 0) || (tahun %% 400 == 0)) {
    kabisat <- TRUE
  } else {
    kabisat <- FALSE
  }

  #Menentukan hari maksimal per bulan
  if (bulan %in% c(1, 3, 5, 7, 8, 10, 12)) {
    maks <- 31
  } else if (bulan %in% c(4, 6, 9, 11)) {
    maks <- 30
  } else if (bulan == 2 && kabisat) {
    maks <- 29
  } else if (bulan == 2 && !kabisat) {
    maks <- 28
  } else {
    maks <- 0 # bulan tidak valid
  }

  #Pengkategorian bulan
  if (bulan < 1 || bulan > 12) {
    cat("Bulan tidak valid!\n")

    # Validasi tanggal
  } else if (tanggal < 1 || tanggal > maks) {
    cat("Tanggal tidak valid! Bulan ini maksimal", maks, "hari.\n")

  #mengganti bulan dari numeric ke character
  } else {
    if (bulan == 1) {
      nama_bulan <- "Januari"
    } else if (bulan == 2) {
      nama_bulan <- "Februari"
    } else if (bulan == 3) {
      nama_bulan <- "Maret"
    } else if (bulan == 4) {
      nama_bulan <- "April"
    } else if (bulan == 5) {

```

```

        nama_bulan <- "Mei"
    } else if (bulan == 6) {
        nama_bulan <- "Juni"
    } else if (bulan == 7) {
        nama_bulan <- "Juli"
    } else if (bulan == 8) {
        nama_bulan <- "Agustus"
    } else if (bulan == 9) {
        nama_bulan <- "September"
    } else if (bulan == 10) {
        nama_bulan <- "Oktober"
    } else if (bulan == 11) {
        nama_bulan <- "November"
    } else if (bulan == 12) {
        nama_bulan <- "Desember"
    }
    cat("Tanggal lahir:", tanggal, nama_bulan, tahun, "\n")
}
}

```

4.2.5 Program 5

```

#Projek 5
#Klasifikasi cluster 3D
#Keadaan 1
#menentukan fungsi jarak
jarak<-function(p,q){
  sqrt((p[1]-q[1])^2+
        (p[2]-q[2])^2+
        (p[3]-q[3])^2)
}
#input pusat dari tiga cluster
A<-c(2,1,3)
B<-c(1,-4,6)
C<-c(-2,3,-2)
#input U
x1<-1
x2<--4
x3<-6

U<-c(x1,x2,x3)
#definisi jarak U ke A, B, C
dA<-jarak(U,A)

```



```

dB<-jarak(U,B)
dC<-jarak(U,C)
#menentukan cluster
if(dA<dB && dA<dC){
  cluster<-"cluster A"
} else if (dB<dA && dB<dC){
  cluster<-"cluster B"
} else if (dC<dA && dC<dB){
  cluster<-"cluster C"
} else if (dA==dB && dA<dC){
  cluster<-"Perbatasan cluster A dan B"
} else if (dA==dC && dA<dB){
  cluster<-"Perbatasan cluster A dan C"
} else if (dB==dC && dB<dA){
  cluster<-"Perbatasan cluster B dan C"
} else {
  cluster<-"Perbatasan cluster A, B, dan C"
}
#Output
cat("Titik termasuk ", cluster, "\n")

```

```

#Keadaan 2
#menentukan fungsi jarak
jarak<-function(p,q){
  sqrt((p[1]-q[1])^2+
        (p[2]-q[2])^2+
        (p[3]-q[3])^2)
}
#input pusat dari tiga cluster
A<-c(2,1,3)
B<-c(1,-4,6)
C<-c(-2,3,-2)
#input U
x1<-4
x2<-2
x3<-3

```

```

U<-c(x1,x2,x3)
#definisi jarak U ke A, B, C
dA<-jarak(U,A)
dB<-jarak(U,B)
dC<-jarak(U,C)
#menentukan cluster
if(dA<dB && dA<dC){

```

```

cluster<-"cluster A"
} else if (dB<dA && dB<dC){
  cluster<-"cluster B"
} else if (dC<dA && dC<dB){
  cluster<-"cluster C"
} else if (dA==dB && dA<dC){
  cluster<-"Perbatasan cluster A dan B"
} else if (dA==dC && dA<dB){
  cluster<-"Perbatasan cluster A dan C"
} else if (dB==dC && dB<dA){
  cluster<-"Perbatasan cluster B dan C"
} else {
  cluster<-"Perbatasan cluster A, B, dan C"
}
#Output
cat("Titik termasuk ", cluster, "\n")

```

```

#Keadaan 3
#menentukan fungsi jarak
jarak<-function(p,q){
  sqrt((p[1]-q[1])^2+
        (p[2]-q[2])^2+
        (p[3]-q[3])^2)
}
#input pusat dari tiga cluster
A<-c(2,1,3)
B<-c(1,-4,6)
C<-c(-2,3,-2)
#input U
x1<-1.5
x2<--1.5
x3<-4.5

```

```

U<-c(x1,x2,x3)
#definisi jarak U ke A, B, C
dA<-jarak(U,A)
dB<-jarak(U,B)
dC<-jarak(U,C)
#menentukan cluster
if(dA<dB && dA<dC){
  cluster<-"cluster A"
} else if (dB<dA && dB<dC){
  cluster<-"cluster B"
} else if (dC<dA && dC<dB){

```

```

cluster<-"cluster C"
} else if (dA==dB && dA<dC){
cluster<-"Perbatasan cluster A dan B"
} else if (dA==dC && dA<dB){
cluster<-"Perbatasan cluster A dan C"
} else if (dB==dC && dB<dA){
cluster<-"Perbatasan cluster B dan C"
} else {
cluster<-"Perbatasan cluster A, B, dan C"
}
#Output
cat("Titik termasuk ", cluster, "\n")

```

4.2.6 Program 6

```

#Program 6
#Keadaan 1
interval_proporsi<-function(phat, n, alpha){
  if(phat<0||phat>1){
    cat("error: proporsi harus antara 0 dan 1\n")
    return()
  }
  if(alpha==0.10){
    z<-1.645
  } else if(alpha==0.05){
    z<-1.96
  } else {
    cat("Error: Nilai alpha harus 0.10 (10%) atau 0.05(5%) \n")
    return()
  }
  margin<-z*sqrt((phat*(1-phat))/n)
  bawah<-phat-margin
  atas <- phat + margin

  cat("Interval Konfidensi:\n")
  cat("(", bawah, ", ", atas, ")\n")
}
phat<-0.3
n <-15
alpha <- 0.05
interval_proporsi(phat,n,alpha)

#Keadaan 2
interval_proporsi<-function(phat, n, alpha){

```

```

if(phat<0||phat>1){
  cat("error: proporsi harus antara 0 dan 1\n")
  return()
}
if(alpha==0.10){
  z<-1.645
}else if(alpha==0.05){
  z<-1.96
}else {
  cat("Error: Nilai alpha harus 0.10 (10%) atau 0.05(5%) \n")
  return()
}
margin<-z*sqrt((phat*(1-phat))/n)
bawah<-phat-margin
atas <- phat + margin

cat("Interval Konfidensi:\n")
cat("(", bawah, ", ", atas, ") \n")
}
phat<-1.5
n <-50
alpha <- 0.10
interval_proporsi(phat,n,alpha)

#Keadaan 3
interval_proporsi<-function(phat, n, alpha){
  if(phat<0||phat>1){
    cat("error: proporsi harus antara 0 dan 1\n")
    return()
  }
  if(alpha==0.10){
    z<-1.645
  }else if(alpha==0.05){
    z<-1.96
  }else {
    cat("Error: Nilai alpha harus 0.10 (10%) atau 0.05(5%) \n")
    return()
  }
  margin<-z*sqrt((phat*(1-phat))/n)
  bawah<-phat-margin
  atas <- phat + margin

  cat("Interval Konfidensi:\n")
  cat("(", bawah, ", ", atas, ") \n")
}

```

```
}  
phat<-0.6  
n <-30  
alpha <- 0.6  
interval_proporsi(phat,n,alpha)
```

BAB V

PENJELASAN LINE-BY-LINE CODE

5.1 Tabel Penjelasan Code R

5.1.1 Program 1

NO	Baris Code	Penjelasan
1	<code>teks<-"...."</code>	Menyimpan sebuah paragraf pada variabel teks.
2	<code>kata <- (strsplit(trimws(teks),"\s+"))[[1]]</code>	Memecah teks menjadi daftar kata berdasarkan spasi. Fungsi trimws digunakan untuk menghapus spasi di awal dan akhir. Setelah itu, kata disimpan pada variabel kata.
3	<code>jumlah_kata <- length(kata)</code>	Menghitung banyak kata dalam variabel kata yang disimpan dalam variabel jumlah_kata.
4	<code>kalimat <- strsplit(teks),"\."</code> [[1]]	Memecah teks menjadi daftar kata berdasarkan titik yang disimpan dalam variabel kalimat.
5	<code>jumlah_kalimat <- length(kalimat)</code>	Menghitung banyak kalimat dalam variabel kalimat yang disimpan dalam variabel jumlah_kalimat.
6	<code>cat(sprintf("Jumlah kalimat:%d kalimat\n", jumlah_kalimat))</code>	Menampilkan jumlah kalimat ke layar.
7	<code>cat(sprintf("Jumlah kata:%d kata\n", jumlah_kata))</code>	Menampilkan jumlah kalimat ke layar.
8	<code>cat(sprintf("Teks tersebut memuat %d kalimat dan %d kata.\n", jumlah_kalimat, jumlah_kata))</code>	Menampilkan jumlah kalimat dan jumlah kata ke layar dalam satu kalimat.

5.1.2 Program 2

NO	Baris Code	Penjelasan
1	<code>K1<-"...."</code>	Menyimpan sebuah kalimat pada variabel K1.

2	K2<-"..."	Menyimpan sebuah kalimat pada variabel K2.
3	K3<-"..."	Menyimpan sebuah kalimat pada variabel K3.
4	K4<-"..."	Menyimpan sebuah kalimat pada variabel K4.
5	cat("K1:", K1, "\n")	Menampilkan kalimat K1 ke layar.
6	cat("K2:", K2, "\n")	Menampilkan kalimat K2 ke layar.
7	cat("K3:", K3, "\n")	Menampilkan kalimat K3 ke layar.
8	cat("K4:", K4, "\n")	Menampilkan kalimat K4 ke layar.

5.1.3 Program 3

NO	Baris Code	Penjelasan
1	a <- ..	Menetapkan nilai a.
2	b <- ..	Menetapkan nilai b.
3	c <- ..	Menetapkan nilai c.
4	$D <- b^2 - 4ac$	Menghitung diskriminan.
5	print(D)	Menampilkan nilai diskriminan.
6	if (D>0) {	Kondisi percabangan 1, ketika D>0 maka persamaan akan memiliki akar yang berbeda.
7	$x1 <- (-b + \sqrt{D}) / (2*a)$	Menghitung akar pertama menggunakan rumus kuadrat.
8	$x2 <- (-b - \sqrt{D}) / (2*a)$	Menghitung akar kedua menggunakan rumus kuadrat.
9	cat(sprintf("x1 = %.3f\n", x1))	Menampilkan nilai akar pertama dengan 3 angka di belakang koma.
10	cat(sprintf("x2 = %.3f\n", x1))	Menampilkan nilai akar kedua dengan 3 angka di belakang koma.
11	} else if (D==0) {	Kondisi percabangan 2, ketika D=0 maka

		persamaan akan memiliki akar kembar.
12	$x <- -b/(2*a)$	Menghitung akar kembar.
13	<code>cat(sprintf("x1 = x2 = %.3f\n", x))</code>	Menampilkan nilai akar kembar dengan 3 angka di belakang koma.
14	<code>} else {</code>	Kondisi percabangan 3, ketika $D < 0$ maka persamaan hanya memiliki akar imajiner.
15	<code>cat("Persamaan hanya memiliki akar imajiner\n")</code>	Menampilkan pesan bahwa persamaan hanya memiliki akar imajiner.
16	<code>}</code>	Menutup fungsi if-else

5.1.4 Program 4

NO	Baris Code	Penjelasan
1	<code>NIP<-...</code>	Menyimpan NIP dalam bentuk string.
2	<code>if (nchar(nip) != 18) {</code>	Memeriksa apakah panjang NIP tepat 18 digit.
3	<code>cat("NIP tidak valid! NIP harus terdiri dari 18 angka.\n")</code>	Menampilkan pesan tidak valid jika panjang NIP tidak tepat 18 digit.
4	<code>} else {</code>	Jika panjang NIP valid, lanjut ke proses selanjutnya.
5	<code>tahun <- as.numeric(substr(nip, 1, 4))</code>	Mengambil 4 digit pertama sebagai tahun.
6	<code>bulan <- as.numeric(substr(nip, 5, 6))</code>	Mengambil digit ke 5 dan 6 sebagai bulan.
7	<code>tanggal <- as.numeric(substr(nip, 7, 8))</code>	Mengambil digit ke 7 dan 8 sebagai tanggal.
8	<code>if ((tahun %% 4 == 0 && tahun %% 100 != 0) (tahun %% 400 == 0)) {</code>	Memeriksa apakah tahun tersebut adalah tahun kabisat.
9	<code>kabisat <- TRUE</code>	Menetapkan bahwa tahun tersebut merupakan tahun kabisat.
10	<code>} else {</code>	Jika kondisi tidak terpenuhi, program menjalankan bagian else.

11	Kabisat<-FALSE	Menetapkan bahwa tahun tersebut bukan merupakan tahun kabisat.
12	}	Menutup percabangan pemeriksaan tahun kabisat.
13	if (bulan %in% c(1, 3, 5, 7, 8, 10, 12)) {	Menentukan jumlah hari maksimum untuk bulan yang memiliki 31 hari.
14	maks <- 31	Menetapkan jumlah maksimum hari sebanyak 31.
15	} else if (bulan %in% c(4, 6, 9, 11)) {	Menentukan jumlah hari maksimum untuk bulan yang memiliki 30 hari.
16	maks <- 30	Menetapkan jumlah maksimum hari sebanyak 30.
17	} else if (bulan == 2 && kabisat) {	Memeriksa february tahun kabisat.
18	maks <- 29	Menetapkan jumlah maksimum hari sebanyak 29
19	} else if (bulan == 2 && !kabisat) {	Memeriksa february pada tahun biasa.
20	maks <- 28	Menetapkan jumlah maksimum hari sebanyak 28.
21	} else {	Jika tidak memenuhi semua kondisi bulan sebelumnya, program menganggap bulan tidak valid.
22	maks <- 0	Menetapkan jumlah maksimum hari 0, karena bulan tidak valid.
23	}	Menutup percabangan penentuan maksimum hari.
24	if (bulan < 1 bulan > 12) {	Memeriksa apakah bulan berada di luar rentang 1 hingga 12.
25	cat("Bulan tidak valid!\n")	Menampilkan pesan bulan tidak valid.
26	} else if (tanggal < 1 tanggal > maks) {	Memeriksa apakah tanggal sesuai dengan jumlah hari maksimum bulan tersebut.
27	cat("Tanggal tidak valid! Bulan ini maksimal", maks, "hari.\n")	Menampilkan pesan tanggal tidak valid.

28	} else {	Jika bulan dan tanggal valid, program melanjutkan proses.
29	if (bulan == 1) {	Memeriksa apakah bulan bernilai 1.
30	nama_bulan <- "Januari"	Mengubah angka 1 menjadi nama bulan Januari.
31	} else if (bulan == 2) {	Memeriksa apakah bulan bernilai 2.
32	nama_bulan <- "Februari"	Mengubah angka 2 menjadi nama bulan Februari.
33	} else if (bulan == 3) {	Memeriksa apakah bulan bernilai 3.
34	nama_bulan <- "Maret"	Mengubah angka 3 menjadi nama bulan Maret.
35	} else if (bulan == 4) {	Memeriksa apakah bulan bernilai 4.
36	nama_bulan <- "April"	Mengubah angka 4 menjadi nama bulan April.
37	} else if (bulan == 5) {	Memeriksa apakah bulan bernilai 5.
38	nama_bulan <- "Mei"	Mengubah angka 5 menjadi nama bulan Mei.
39	} else if (bulan == 6) {	Memeriksa apakah bulan bernilai 6.
40	nama_bulan <- "Juni"	Mengubah angka 6 menjadi nama bulan Juni.
41	} else if (bulan == 7) {	Memeriksa apakah bulan bernilai 7.
42	nama_bulan <- "Juli"	Mengubah angka 7 menjadi nama bulan Juli.
43	} else if (bulan == 8) {	Memeriksa apakah bulan bernilai 8.
44	nama_bulan <- "Agustus"	Mengubah angka 8 menjadi nama bulan Agustus.
45	} else if (bulan == 9) {	Memeriksa apakah bulan bernilai 9.
46	nama_bulan <- "September"	Mengubah angka 9 menjadi nama bulan September.
47	} else if (bulan == 10) {	Memeriksa apakah bulan bernilai 10.

48	<code>nama_bulan <- "Oktober"</code>	Mengubah angka 10 menjadi nama bulan Oktober.
49	<code>} else if (bulan == 11) {</code>	Memeriksa apakah bulan bernilai 11.
50	<code>nama_bulan <- "November"</code>	Mengubah angka 11 menjadi nama bulan November.
51	<code>} else if (bulan == 12) {</code>	Memeriksa apakah bulan bernilai 12.
52	<code>nama_bulan <- "Desember"</code>	Mengubah angka 12 menjadi nama bulan Desember.
53	<code>}</code>	Menutup percabangan konversi bulan.
54	<code>cat("Tanggal lahir:", tanggal, nama_bulan, tahun, "\n")</code>	Menampilkan tanggal lahir dalam format tanggal, nama bulan, dan tahun.
55	<code>}</code>	Menutup blok validasi bulan dan tanggal.
56	<code>}</code>	Menutup blok validasi panjang NIP.

5.1.5 Program 5

NO	Baris Code	Penjelasan
1	<code>jarak<-function(p,q){</code>	Mendefinisikan fungsi bernama jarak dengan parameter p dan q
2	<code>sqrt((p[1]-q[1])^2+</code>	Menghitung kuadrat selisih koordinat pertama dan mulai menghitung jarak Euclid.
3	<code>(p[2]-q[2])^2+</code>	Menghitung kuadrat selisih koordinat kedua.
4	<code>(p[3]-q[3])^2)</code>	Menghitung kuadrat selisih koordinat ketiga.
5	<code>}</code>	Menutup fungsi jarak.
6	<code>A<-c(2,1,3)</code>	Mendefinisikan pusat kluster A.
7	<code>B<-c(1,-4,6)</code>	Mendefinisikan pusat kluster B.
8	<code>C<-c(-2,3,-2)</code>	Mendefinisikan pusat kluster C.
9	<code>x1<-1</code>	Menyimpan nilai koordinat pertama titik U.

10	$x_2 \leftarrow -4$	Menyimpan nilai koordinat Kedua titik U.
11	$x_3 \leftarrow -6$	Menyimpan nilai koordinat ketiga titik U.
12	$U \leftarrow c(x_1, x_2, x_3)$	Membuat vektor U dengan koordinat x_1 , x_2 , dan x_3 .
13	$d_A \leftarrow \text{jarak}(U, A)$	Menghitung jarak titik U ke pusat cluster A.
14	$d_B \leftarrow \text{jarak}(U, B)$	Menghitung jarak titik U ke pusat cluster B
15	$d_C \leftarrow \text{jarak}(U, C)$	Menghitung jarak titik U ke pusat cluster C.
16	$\text{if}(d_A < d_B \ \&\& \ d_A < d_C)\{$	Kondisi percabangan 1, ketika jarak ke A paling dekat maka termasuk ke titik A.
17	$\text{cluster} \leftarrow \text{"cluster A"}$	Menentukan titik berada di cluster A.
18	$\} \text{ else if } (d_B < d_A \ \&\& \ d_B < d_C)\{$	Kondisi percabangan 2, ketika jarak ke B paling dekat maka termasuk ke titik B.
19	$\text{cluster} \leftarrow \text{"cluster B"}$	Menentukan titik berada di cluster B.
20	$\} \text{ else if } (d_C < d_A \ \&\& \ d_C < d_B)\{$	Kondisi percabangan 3, ketika jarak ke C paling dekat maka termasuk ke titik C.
21	$\text{cluster} \leftarrow \text{"cluster C"}$	Menentukan titik berada di cluster C
22	$\} \text{ else if } (d_A == d_B \ \&\& \ d_A < d_C)\{$	Kondisi percabangan 4, ketika jarak ke A sama dengan jarak B dan lebih kecil dari jarak ke C maka termasuk ke perbatasan A dan B.
23	$\text{cluster} \leftarrow \text{"Perbatasan cluster A dan B"}$	Menentukan titik berada di perbatasan cluster A dan B.
24	$\} \text{ else if } (d_A == d_C \ \&\& \ d_A < d_B)\{$	Kondisi percabangan 5, ketika jarak ke A sama dengan jarak C dan lebih kecil dari jarak ke B maka termasuk ke perbatasan A dan C.

25	<code>cluster<-"Perbatasan cluster A dan C"</code>	Menentukan titik berada di perbatasan cluster A dan C.
26	<code>} else if (dB==dC && dB<dA){</code>	Kondisi percabangan 6, ketika jarak ke B sama dengan jarak C dan lebih kecil dari jarak ke A maka termasuk ke perbatasan B dan C.
27	<code>cluster<-"Perbatasan cluster B dan C"</code>	Menentukan titik berada di perbatasan cluster B dan C.
28	<code>} else {</code>	Kondisi percabangan 7, ketika 6 kondisi sebelumnya tidak terpenuhi maka termasuk ke perbatasan ketiga cluster.
29	<code>cluster<-"Perbatasan cluster A, B, dan C"</code>	Menentukan titik berada di perbatasan ketiga cluster.
30	<code>}</code>	Menutup fungsi if-else
31	<code>cat("Titik termasuk ", cluster, "\n")</code>	Menampilkan hasil cluster tempat titik U berada.

5.1.6 Program 6

NO	Baris Code	Penjelasan
1	<code>interval_proporsi<-function(phat, n, alpha){</code>	Mendefinisikan fungsi bernama interval_proporsi dengan parameter phat, n, dan alpha.
2	<code>if(phat<0 phat>1){</code>	Memeriksa apakah nilai proporsi sampel (phat) berada di luar rentang yang valid, yaitu kurang dari 0 atau lebih dari 1. Operator <code> </code> berarti "atau", sehingga kondisi akan bernilai benar jika salah satu syarat terpenuhi.
3	<code>cat("error: proporsi harus antara 0 dan 1\n")</code>	Menampilkan pesan kesalahan jika proporsi tidak valid.

4	<code>return()</code>	Menghentikan eksekusi fungsi.
5	<code>{</code>	Menutup blok if.
6	<code>if(alpha==0.10){</code>	Memeriksa apakah nilai alpha adalah 10%.
7	<code>z<-1.645</code>	Menetapkan nilai $Z=1.645$ untuk tingkat kepercayaan 90%.
8	<code>} else if(alpha==0.05){</code>	Memeriksa apakah nilai alpha adalah 5%.
9	<code>z<-1.96</code>	Menetapkan nilai $Z=1.96$ untuk tingkat kepercayaan 95%.
10	<code>} else {</code>	Jika nilai alpha bukan 0.10 maupun 0.05.
11	<code>cat("Error: Nilai alpha harus 0.10 (10%) atau 0.05(5%) \n")</code>	Menampilkan pesan eror.
12	<code>return()</code>	Menghentikan fungsi.
13	<code>}</code>	Menutup blok pemilihan alpha.
14	<code>margin<-zsqrt((phat(1-phat))/n)</code>	Menghitung margin of eror interval kepercayaan.
15	<code>bawah<-phat-margin</code>	Menghitung batas bawah interval kepercayaan.
16	<code>atas <- phat + margin</code>	Menghitung batas atas interval kepercayaan.
17	<code>cat("Interval Konfidensi:\n")</code>	Menampilkan judul output.
18	<code>cat("(", bawah, ",", atas, ")\n")</code>	Menampilkan interval kepercayaan.
19	<code>}</code>	Menutup fungsi interval_proporsi.
20	<code>phat<-0.3</code>	Menetapkan nilai phat sebesar 0.3.
21	<code>n <-15</code>	Menetapkan nilai n sebanyak 15.
22	<code>alpha <- 0.05</code>	Menetapkan nilai alpha sebesar 0.05.
23	<code>interval_proporsi(phat,n,alpha)</code>	Memanggil fungsi untuk menghitung interval kepercayaan proporsi.

5.2 Tabel Penjelasan Code Python

5.2.1 Program 1

NO	Baris Code	Penjelasan
1	Teks="...."	Menyimpan sebuah paragraf pada variabel teks.
2	banyaknya_kalimat = teks.count('.')	Menghitung jumlah titik pada paragraf yang dianggap sebagai akhir kalimat.
3	banyaknya_kata= len(teks.split())	Menghitung banyak kata dengan memisahkan teks berdasarkan spasi.
4	print('Banyaknya Kalimat adalah',banyaknya_kalimat)	Menampilkan jumlah kalimat.
5	print('Banyaknya Kata adalah',banyaknya_kata)	Menampilkan jumlah kata.

5.2.2 Program 2

NO	Baris Code	Penjelasan
1	K1="...."	Menyimpan sebuah kalimat pada variabel K1.
2	K2="...."	Menyimpan sebuah kalimat pada variabel K2.
3	K3="...."	Menyimpan sebuah kalimat pada variabel K3.
4	K4="...."	Menyimpan sebuah kalimat pada variabel K4.
5	Print(K1)	Menampilkan kalimat K1 ke layar.
6	Print(K2)	Menampilkan kalimat K2 ke layar.
7	Print(K3)	Menampilkan kalimat K3 ke layar.
8	Print(K4)	Menampilkan kalimat K4 ke layar.

5.2.3 Program 3

NO	Baris Code	Penjelasan
1	import math	Mengimpor modul math untuk menggunakan fungsi matematika, seperti akar kuadrat (sqrt).

2	<code>a = ..</code>	Menetapkan nilai a.
3	<code>b = ..</code>	Menetapkan nilai b.
4	<code>c = ..</code>	Menetapkan nilai c.
5	<code>d=b**4*a*c</code>	Menghitung diskriminan.
6	<code>if d>0:</code>	Kondisi percabangan 1, ketika $D>0$ maka persamaan akan memiliki akar yang berbeda.
7	<code>x1 = (-b + math.sqrt(d))/(2*a)</code>	Menghitung akar pertama menggunakan rumus kuadrat.
8	<code>x2 = (-b - math.sqrt(d))/(2*a)</code>	Menghitung akar kedua menggunakan rumus kuadrat.
9	<code>print("Akar-akarnya adalah",x1,"dan",x2)</code>	Menampilkan nilai akar x1 dan x2.
10	<code>elif d == 0:</code>	Kondisi percabangan 2, ketika $D=0$ maka persamaan akan memiliki akar kembar.
11	<code>x = -b/(2*a)</code>	Menghitung akar kembar.
12	<code>print("Akar-akarnya adalah",x)</code>	Menampilkan nilai akar kembar
13	<code>else:</code>	Kondisi percabangan 3, ketika $D<0$ maka persamaan hanya memiliki akar imajiner.
14	<code>print("Persamaan hanya memiliki akar imajiner")</code>	Menampilkan pesan bahwa persamaan hanya memiliki akar imajiner.

5.2.4 Program 4

NO	Baris Code	Penjelasan
1	<code>NIP=str(...)</code>	Menyimpan NIP dalam bentuk string.
2	<code>tahun = int(nip[0:4])</code>	Mengambil 4 digit pertama sebagai tahun.
3	<code>bulan = int(nip[4:6])</code>	Mengambil digit ke 5 dan 6 sebagai bulan.
4	<code>tanggal = int(nip[6:8])</code>	Mengambil digit ke 7 dan 8 sebagai tanggal.
5	<code>Kabisat = False</code>	Menginisialisasi bahwa tahun bukan tahun kabisat.

6	If tahun % 4 == 0:	Memeriksa apakah tahun habis dibagi 4.
7	kabisat = True	Menetapkan bahwa tahun tersebut merupakan tahun kabisat.
8	hari_maks = 0	Menginisialisasi jumlah hari maksimum dalam suatu bulan.
9	is_valid_month = True	Menandai bahwa bulan dianggap valid.
10	if bulan in [1, 3, 5, 7, 8, 10, 12]:	Menentukan jumlah hari maksimum untuk bulan yang memiliki 31 hari.
11	hari_maks = 31	Menetapkan jumlah maksimum hari sebanyak 31.
12	elif bulan in [4, 6, 9, 11]:	Menentukan jumlah hari maksimum untuk bulan yang memiliki 30 hari.
13	hari_maks = 30	Menetapkan jumlah maksimum hari sebanyak 30.
14	elif bulan == 2	Memeriksa februari
15	If kabisat:	Memeriksa februari pada tahun kabisat.
16	hari_maks = 29	Menetapkan jumlah maksimum hari sebanyak 29.
17	else:	Memeriksa februari pada tahun biasa.
18	hari_maks = 28	Menetapkan jumlah maksimum hari sebanyak 28.
19	else :	Memeriksa jika bulan tidak valid.
20	is_valid_month = False	Menandai bulan tidak valid.
21	nama_bulan = "Bulan tidak valid"	Menyimpan keterangan bulan tidak valid.
22	if not is_valid_month:	Memeriksa apakah bulan tidak valid.
23	print(f"Bulan {bulan} tidak valid.")	Menampilkan pesan bulan tidak valid.
24	elif tanggal < 1 or tanggal > hari_maks:	Memeriksa apakah tanggal sesuai dengan jumlah hari maksimum bulan tersebut.

25	<code>print("Tanggal lahir tidak valid")</code>	Menampilkan pesan “tanggal lahir tidak valid” jika tanggal tidak valid.
26	<code>else:</code>	Jika bulan dan tanggal valid, program melanjutkan proses.
27	<code>If bulan == 1:</code>	Memeriksa apakah bulan bernilai 1.
28	<code>nama_bulan = "Januari"</code>	Mengubah angka 1 menjadi nama bulan Januari.
29	<code>elif bulan == 2:</code>	Memeriksa apakah bulan bernilai 2.
30	<code>nama_bulan = "Februari"</code>	Mengubah angka 2 menjadi nama bulan Februari.
31	<code>else if bulan == 3:</code>	Memeriksa apakah bulan bernilai 3.
32	<code>nama_bulan = "Maret"</code>	Mengubah angka 3 menjadi nama bulan Maret.
33	<code>elif bulan == 4:</code>	Memeriksa apakah bulan bernilai 4.
34	<code>nama_bulan = "April"</code>	Mengubah angka 4 menjadi nama bulan April.
35	<code>elif bulan == 5:</code>	Memeriksa apakah bulan bernilai 5.
36	<code>nama_bulan = "Mei"</code>	Mengubah angka 5 menjadi nama bulan Mei.
37	<code>elif bulan == 6:</code>	Memeriksa apakah bulan bernilai 6.
38	<code>nama_bulan = "Juni"</code>	Mengubah angka 6 menjadi nama bulan Juni.
39	<code>elif bulan == 7:</code>	Memeriksa apakah bulan bernilai 7.
40	<code>nama_bulan = "Juli"</code>	Mengubah angka 7 menjadi nama bulan Juli.
41	<code>elif bulan == 8:</code>	Memeriksa apakah bulan bernilai 8.
42	<code>nama_bulan = "Agustus"</code>	Mengubah angka 8 menjadi nama bulan Agustus.
43	<code>elif bulan == 9:</code>	Memeriksa apakah bulan bernilai 9.
44	<code>nama_bulan = "September"</code>	Mengubah angka 9 menjadi nama bulan September.

45	<code>elif bulan == 10:</code>	Memeriksa apakah bulan bernilai 10.
46	<code>nama_bulan = "Oktober"</code>	Mengubah angka 10 menjadi nama bulan Oktober.
47	<code>elif bulan == 11:</code>	Memeriksa apakah bulan bernilai 11.
48	<code>nama_bulan = "November"</code>	Mengubah angka 11 menjadi nama bulan November.
49	<code>elif bulan == 12:</code>	Memeriksa apakah bulan bernilai 12.
50	<code>nama_bulan = "Desember"</code>	Mengubah angka 12 menjadi nama bulan Desember.
51	<code>print(f'Tanggal lahir: {tanggal} {nama_bulan} {tahun}.')</code>	Menampilkan tanggal lahir dalam format tanggal, nama bulan, dan tahun.

5.2.5 Program 5

NO	Baris Code	Penjelasan
1	<code>import math</code>	Mengimpor modul <code>math</code> untuk menggunakan fungsi matematika, seperti akar kuadrat (<code>sqrt</code>).
2	<code>A=(2,1,3)</code>	Mendefinisikan pusat kluster A.
3	<code>B=(1,-4,6)</code>	Mendefinisikan pusat kluster B.
4	<code>C=(-2,3,-2)</code>	Mendefinisikan pusat kluster C.
5	<code>x1=1</code>	Menyimpan nilai koordinat pertama titik U.
6	<code>x2=-4</code>	Menyimpan nilai koordinat kedua titik U.
7	<code>x3=6</code>	Menyimpan nilai koordinat ketiga titik U.
8	<code>U=(x1,x2,x3)</code>	Membuat vektor U dengan koordinat <code>x1</code> , <code>x2</code> , dan <code>x3</code> .
9	<code>def jarak(p, q)</code>	Mendefinisikan fungsi bernama <code>jarak</code> dengan parameter <code>p</code> dan <code>q</code> .

10	$\text{hasil_jarak} = \text{math.sqrt}((p[0]-q[0])**2 + (p[1]-q[1])**2 + (p[2]-q[2])**2)$	Menghitung jarak Euclidean antara dua titik tiga dimensi.
11	return hasil_jarak	Mengembalikan hasil perhitungan jarak.
12	$dA = \text{jarak}(U,A)$	Menghitung jarak titik U ke pusat cluster A.
13	$dB = \text{jarak}(U,B)$	Menghitung jarak titik U ke pusat cluster B.
14	$dC = \text{jarak}(U,C)$	Menghitung jarak titik U ke pusat cluster C.
15	if $dA < dB$ and $dA < dC$:	Kondisi percabangan 1, ketika jarak ke A paling dekat maka termasuk ke titik A.
16	cluster="cluster A"	Menentukan titik berada di cluster A.
17	elif $dB < dA$ and $dB < dC$:	Kondisi percabangan 2, ketika jarak ke B paling dekat maka termasuk ke titik B.
18	cluster="cluster B"	Menentukan titik berada di cluster B.
19	elif $dC < dA$ and $dC < dB$:	Kondisi percabangan 3, ketika jarak ke C paling dekat maka termasuk ke titik C.
20	cluster = "cluster C"	Menentukan titik berada di cluster C.
21	elif $dA == dB$ and $dA < dC$:	Kondisi percabangan 4, ketika jarak ke A sama dengan jarak B dan lebih kecil dari jarak ke C maka termasuk ke perbatasan A dan B.
22	cluster="Perbatasan cluster A dan B"	Menentukan titik berada di perbatasan cluster A dan B.
23	elif $dA == dC$ and $dA < dB$:	Kondisi percabangan 5, ketika jarak ke A sama dengan jarak C dan lebih kecil dari jarak ke B maka termasuk ke perbatasan A dan C.
24	cluster="Perbatasan cluster A dan C"	Menentukan titik berada di perbatasan cluster A dan C.

25	<code>elif dB==dC and dB<dA:</code>	Kondisi percabangan 6, ketika jarak ke B sama dengan jarak C dan lebih kecil dari jarak ke A maka termasuk ke perbatasan B dan C.
26	<code>cluster="Perbatasan cluster B dan C"</code>	Menentukan titik berada di perbatasan cluster B dan C.
27	<code>else:</code>	Kondisi percabangan 7, ketika 6 kondisi sebelumnya tidak terpenuhi maka termasuk ke perbatasan ketiga cluster.
28	<code>cluster="Perbatasan cluster A, B, dan C"</code>	Menentukan titik berada di perbatasan ketiga cluster.
29	<code>print(f'Koordinat titik U : {U}')</code>	Menampilkan koordinat titik U.
30	<code>print(f'Jarak U ke A : {dA:.4f}')</code>	Menampilkan jarak U ke A dengan 4 angka di belakang koma.
31	<code>print(f'Jarak U ke B : {dB:.4f}')</code>	Menampilkan jarak U ke B dengan 4 angka di belakang koma.
32	<code>print(f'Jarak U ke C : {dC:.4f}')</code>	Menampilkan jarak U ke C dengan 4 angka di belakang koma.
33	<code>print(f'Titik tergolong {cluster}')</code>	Menampilkan hasil cluster tempat titik U berada.

5.2.6 Program 6

NO	Baris Code	Penjelasan
1	<code>import math</code>	Mengimpor modul math untuk menggunakan fungsi matematika, seperti akar kuadrat (sqrt).
2	<code>def interval_konfidensi(p_hat, alpha, n):</code>	Mendefinisikan fungsi bernama interval_konfidensi dengan parameter phat, n, dan alpha.
3	<code>if p_hat < 0 or p_hat > 1:</code>	Memeriksa apakah nilai proporsi sampel (phat)

		berada di luar rentang yang valid, yaitu kurang dari 0 atau lebih dari 1. Operator or berarti kondisi akan bernilai benar jika salah satu syarat terpenuhi.
4	<code>print("Error: Proporsi harus antara 0 dan 1.")</code>	Menampilkan pesan eror.
5	<code>return</code>	Menghentikan eksekusi fungsi.
6	<code>If alpha==0.10:</code>	Memeriksa apakah nilai alpha adalah 10%.
7	<code>z=1.645</code>	Menetapkan nilai $Z=1.645$ untuk tingkat kepercayaan 90%.
8	<code>Elif alpha==0.05:</code>	Memeriksa apakah nilai alpha adalah 5%.
9	<code>z=1.96</code>	Menetapkan nilai $Z=1.96$ untuk tingkat kepercayaan 95%.
10	<code>else:</code>	Jika nilai alpha bukan 0.10 maupun 0.05.
11	<code>print("Error: Nilai alpha harus 0.10 (10%) atau 0.05(5%) .")</code>	Menampilkan pesan eror.
12	<code>return</code>	Menghentikan eksekusi fungsi
13	<code>margin = z * math.sqrt((p_hat * (1 - p_hat)) / n)</code>	Menghitung margin of eror interval kepercayaan.
14	<code>batas_bawah = p_hat - margin</code>	Menghitung batas bawah interval kepercayaan.
15	<code>batas_atas = p_hat + margin</code>	Menghitung batas atas interval kepercayaan.
16	<code>print(f"Interval konfidensi : ({batas_bawah:.4f}, {batas_atas:.4f})")</code>	Menampilkan interval konfidensi dengan 4 angka di belakang koma.
17	<code>print(f"Kesimpulan: {batas_bawah:.4f} < p < {batas_atas:.4f}")</code>	Menampilkan kesimpulan bahwa proporsi populasi berada di antara batas bawah dan batas atas interval konfidensi.
18	<code>interval_konfidensi(0.3, 0.05, 15)</code>	Memanggil fungsi dengan nilai proporsi

		0,3, alpha 0,05, dan ukuran sampel 15.
--	--	---

BAB VI

PENGUJIAN PROGRAM

Setiap program diuji dengan tiga skenario untuk membuktikan berhasil atau tidaknya program tersebut. Adapun cakupan pengujian yaitu input normal dan input tidak valid (hanya jika relevan). Berikut merupakan tabel pengujian, screenshot hasil running, dan interpretasi output, serta perbaikan bug jika ada.

6.1 Tabel Pengujian

6.1.1 Program 1

Skenario Uji	Input	Output yang diharapkan	Output aktual	Status	Catatan
Keadaan 1	Teks Informal bertema kehidupan mahasiswa statistika	Program menampilkan jumlah kalimat dan jumlah kata sesuai isi teks	Teks memuat 3 kalimat dan 49 kata	Berhasil	Pengujian kondisi normal dengan 3 kalimat.
Keadaan 2	Teks formal bertema olahraga	Program menampilkan jumlah kalimat dan kata sesuai isi teks	Teks memuat 7 kalimat dan 84 kata	Berhasil	Menguji teks yang lebih panjang dengan lebih banyak kalimat.
Keadaan 3	Teks formal bertema kuliner	Program tetap menghitung berdasarkan tanda titik (.)	Teks memuat 9 kalimat dan 93 kata	Berhasil	Menguji kondisi khusus karena terdapat dua tanda titik berurutan sehingga menghasilkan elemen kosong saat strsplit()

6.1.2 Program 2

Skenario Uji	Input	Output yang diharapkan	Output aktual	Status	Catatan
1	String K1-K4	Seluruh string ditampilkan sesuai soal, termasuk tanda ('), tanda ("), karakter <i>escape</i> , angka, titik, dan garis miring.	K1, K2, K3, dan K4 berhasil ditampilkan sesuai dengan format yang diminta.	Berhasil	Program tidak menerima input dari pengguna sehingga pengujian hanya dilakukan sekali untuk memastikan

					seluruh string tampil dengan benar.
--	--	--	--	--	-------------------------------------

6.1.3 Program 3

Skenario Uji	Input (a, b, c)	Output yang diharapkan	Output aktual	Status	Catatan
Keadaan 1	(1, 4, 4)	Akar-akarnya adalah $x_1 = x_2 = -2$	Akar-akarnya adalah $x_1 = x_2 = -2$	Berhasil	Output merupakan akar kembar
Keadaan 2	(2, 3, 6)	Persamaan hanya memiliki akar imajiner	Persamaan hanya memiliki akar imajiner	Berhasil	Diskriminan < 0
Keadaan 3	(1, -5, 6)	Akar-akarnya adalah $x_1 = 3$ dan $x_2 = 2$	Akar-akarnya adalah $x_1 = 3$ dan $x_2 = 2$	Berhasil	Dua akar real berbeda

6.1.4 Program 4

Skenario Uji	Input NIP	Output yang diharapkan	Output aktual	Status	Catatan
Keadaan 1	199904212019031010	Tanggal Lahir 21 April 1999	Tanggal Lahir 21 April 1999	Berhasil	NIP yang diinput 18 digit dan sesuai dengan batas yang ada
Keadaan 2	199609162019031008	Tanggal lahir: 16 September 1996	Tanggal lahir: 16 September 1996	Berhasil	NIP yang diinput 18 digit dan sesuai dengan batas yang ada
Keadaan 3	199513152019031006	Tanggal lahir: 15 Bulan tidak Valid 1995	Tanggal lahir: 15 Bulan tidak Valid 1995	Berhasil	Bulan melebihi batas (13>12) sehingga tidak valid.

6.1.5 Program 5

Skenario Uji	Input $U(x_1, x_2, x_3)$	Output yang diharapkan	Output aktual	Status	Catatan
Keadaan 1	(1, -4, 6)	Titik termasuk cluster B	Titik termasuk cluster B	Berhasil	Jarak B merupakan jarak terkecil sehingga titik termasuk cluster B
Keadaan 2	(4, 2, 3)	Titik termasuk cluster A	Titik termasuk cluster A	Berhasil	Jarak A merupakan jarak terkecil sehingga titik termasuk cluster A
Keadaan 3	(1.5, -1.5, 4.5)	Titik termasuk perbatasan cluster A dan B	Titik termasuk perbatasan cluster A dan B	Berhasil	Jarak A== Jarak B, sehingga titik termasuk perbatasan cluster A dan B

6.1.6 Program 6

Skenario Uji	Input (p, n, α)	Output yang diharapkan	Output aktual	Status	Catatan
Keadaan 1	(0.3, 15, 0.05)	Interval konfidensi: (0.0681, 0.5319)	Interval konfidensi: (0.0681, 0.5319)	Berhasil	Validasi lolos, $z = 1.96$, margin dihitung
Keadaan 2	(1.5, 50, 0.10)	Error: Proporsi harus antara 0 dan 1.	Error: Proporsi harus antara 0 dan 1.	Berhasil	Validasi p gagal ($p > 1$)
Keadaan 3	(0.6, 30, 0.6)	Error: Nilai alpha harus 0.10 (10%) atau 0.05 (5%).	Error: Nilai alpha harus 0.10 (10%) atau 0.05 (5%).	Berhasil	Validasi p lolos, Validasi α gagal

6.2 Screenshot Hasil Running

6.2.1 Python

1. Program 1
 - a) Keadaan 1

```
#Program 1 Menghitung banyaknya kata dan banyaknya kalimat
#Keadaan 1
teks= "Kehidupan anak statistika tidak jauh dari data, angka,

# Menghitung banyaknya kalimat dan banyaknya kata
banyaknya_kalimat = teks.count('.')
banyaknya_kata= len(teks.split())
print('Banyaknya Kalimat adalah',banyaknya_kalimat)
print('Banyaknya Kata adalah',banyaknya_kata)
```

Banyaknya Kalimat adalah 3
Banyaknya Kata adalah 49

b) Keadaan 2

```
#Keadaan 2
teks= ""Olahraga merupakan aktivitas fisik yang sangat penting

# Menghitung banyaknya kalimat dan banyaknya kata
banyaknya_kalimat = teks.count('.')
banyaknya_kata= len(teks.split())
print('Banyaknya Kalimat adalah',banyaknya_kalimat)
print('Banyaknya Kata adalah',banyaknya_kata)
```

... Banyaknya Kalimat adalah 7
Banyaknya Kata adalah 84

c) Keadaan 3

```
#Keadaan 3
teks= ""Indonesia memiliki kekayaan kuliner tradisional yang

# Menghitung banyaknya kalimat dan banyaknya kata
banyaknya_kalimat = teks.count('.')
banyaknya_kata= len(teks.split())
print('Banyaknya Kalimat adalah',banyaknya_kalimat)
print('Banyaknya Kata adalah',banyaknya_kata)
```

... Banyaknya Kalimat adalah 9
Banyaknya Kata adalah 93

2. Program 2

```
#Program 2 Menampilkan Kalimat
K1= "Saya tak 'kan menyerah"
K2= "Ia berkata,\"Aku Menyayangimu\"."
K3= "\"Coba jelaskan pengertian 'cross-validation' dalam Machi
K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."

print(K1)
print(K2)
print(K3)
print(K4)
```

```
... Saya tak 'kan menyerah
Ia berkata,"Aku Menyayangimu".
"Coba jelaskan pengertian 'cross-validation' dalam Machine Lear
Surat keputusan itu bernomor 62/UN.34/19/2023.
```

3. Program 3

a) Keadaan 1

```
#Program 3 Mencari Akar-Akar Persamaan Kuadrat  $ax^2 + bx + c = 0$ 
#Kondisi 1
import math
a = 1
b = 4
c = 4
d = ( b**2 - 4*a*c)
if d>0:
    x1 = (-b + math.sqrt(d))/(2*a)
    x2 = (-b - math.sqrt(d))/(2*a)
    print("Akar-akarnya adalah",x1,"dan",x2)
elif d == 0:
    x = -b/(2*a)
    print("Akar-akarnya adalah",x)
else:
    print("Persamaan hanya memiliki akar imajiner")
```

```
... Akar-akarnya adalah -2.0
```

[How can I install Python libraries?](#) [Load da](#)

b) Keadaan 2

```
#Kondisi 2
import math
a = 2
b = 3
c = 6
d = ( b**2 - 4*a*c)
if d>0:
    x1 = (-b + math.sqrt(d))/(2*a)
    x2 = (-b - math.sqrt(d))/(2*a)
    print("Akar-akarnya adalah",x1,"dan",x2)
elif d == 0:
    x = -b/(2*a)
    print("Akar-akarnya adalah",x)
else:
    print("Persamaan hanya memiliki akar imajiner")
```

```
... Persamaan hanya memiliki akar imajiner
```

c) Kondisi 3

```
#Kondisi 3
import math
a = 1
b = -5
c = 6
d = ( b**2 - 4*a*c)
if d>0:
    x1 = (-b + math.sqrt(d))/(2*a)
    x2 = (-b - math.sqrt(d))/(2*a)
    print("Akar-akarnya adalah",x1,"dan",x2)
elif d == 0:
    x = -b/(2*a)
    print("Akar-akarnya adalah",x)
else:
    print("Persamaan hanya memiliki akar imajiner")
```

*** Akar-akarnya adalah 3.0 dan 2.0

4. Program 4

a) Keadaan 1

```
nama_bulan = "Juni"
elif bulan == 7:
    nama_bulan = "Juli"
elif bulan == 8:
    nama_bulan = "Agustus"
elif bulan == 9:
    nama_bulan = "September"
elif bulan == 10:
    nama_bulan = "Oktober"
elif bulan == 11:
    nama_bulan = "November"
elif bulan == 12:
    nama_bulan = "Desember"

print(f"Tanggal lahir: {tanggal} {nama_bulan} {tahun}.")
```

*** Tanggal lahir: 21 April 1999.

b) Keadaan 2

```
nama_bulan = "Mei"
elif bulan == 6:
    nama_bulan = "Juni"
elif bulan == 7:
    nama_bulan = "Juli"
elif bulan == 8:
    nama_bulan = "Agustus"
elif bulan == 9:
    nama_bulan = "September"
elif bulan == 10:
    nama_bulan = "Oktober"
elif bulan == 11:
    nama_bulan = "November"
elif bulan == 12:
    nama_bulan = "Desember"

print(f"Tanggal lahir: {tanggal} {nama_bulan} {tahun}.")
```

Tanggal lahir: 16 September 1996.

c) Keadaan 3

```

nama_bulan = "Juni"
elif bulan == 7:
    nama_bulan = "Juli"
elif bulan == 8:
    nama_bulan = "Agustus"
elif bulan == 9:
    nama_bulan = "September"
elif bulan == 10:
    nama_bulan = "Oktober"
elif bulan == 11:
    nama_bulan = "November"
elif bulan == 12:
    nama_bulan = "Desember"

print(f"Tanggal lahir: {tanggal} {nama_bulan} {tahun}.")

*** Bulan 13 tidak valid.

```

5. Program 5

a) Keadaan 1

```

if dA==dC and dA<dB:
    cluster="Perbatasan cluster A dan C"
if dB==dC and dB<dA:
    cluster="Perbatasan cluster B dan C"
se:
    cluster="Perbatasan cluster A, B, dan C"

#Tampilkan hasil
int(f"Koordinat titik U : {U}")
int(f"Jarak U ke A : {dA:.4f}")
int(f"Jarak U ke B : {dB:.4f}")
int(f"Jarak U ke C : {dC:.4f}")
int(f"Titik tergolong {cluster}")

*** Koordinat titik U : (1, -4, 6)
Jarak U ke A : 5.9161
Jarak U ke B : 0.0000
Jarak U ke C : 11.0454
Titik tergolong cluster B

```

b) Keadaan 2

```

elif dA==dC and dA<dB:
    cluster="Perbatasan cluster A dan C"
elif dB==dC and dB<dA:
    cluster="Perbatasan cluster B dan C"
else:
    cluster="Perbatasan cluster A, B, dan C"

#Tampilkan hasil
print(f"Koordinat titik U : {U}")
print(f"Jarak U ke A : {dA:.4f}")
print(f"Jarak U ke B : {dB:.4f}")
print(f"Jarak U ke C : {dC:.4f}")
print(f"Titik tergolong {cluster}")

*** Koordinat titik U : (4, 2, 3)
Jarak U ke A : 2.2361
Jarak U ke B : 7.3485
Jarak U ke C : 7.8740
Titik tergolong cluster A

```

c) Keadaan 3

```

elif dA==dC and dA<dB:
    cluster="Perbatasan cluster A dan C"
elif dB==dC and dB<dA:
    cluster="Perbatasan cluster B dan C"
else:
    cluster="Perbatasan cluster A, B, dan C"

#Tampilkan hasil
print(f"Koordinat titik U : {U}")
print(f"Jarak U ke A : {dA:.4f}")
print(f"Jarak U ke B : {dB:.4f}")
print(f"Jarak U ke C : {dC:.4f}")
print(f"Titik tergolong {cluster}")

```

```

*** Koordinat titik U : (1.5, -1.5, 4.5)
    Jarak U ke A : 2.9580
    Jarak U ke B : 2.9580
    Jarak U ke C : 8.6458
    Titik tergolong Perbatasan cluster A dan B

```

6. Program 6

a) Keadaan 1

```

else:
    print("Error: Nilai alpha harus < 0.05")
    return

# Menghitung margin of error
margin = z * math.sqrt((p_hat * (1 - p_hat)) / n)

# Menghitung batas interval
batas_bawah = p_hat - margin
batas_atas = p_hat + margin

# Menampilkan hasil
print(f"Interval konfidensi : ({batas_bawah:.4f}, {batas_atas:.4f})")
print(f"Kesimpulan: {batas_bawah:.4f} < p < {batas_atas:.4f}")

# Memanggil fungsi
interval_konfidensi(0.3, 0.05, 15)

```

```

*** Interval konfidensi : (0.0681, 0.5319)
    Kesimpulan: 0.0681 < p < 0.5319

```

b) Kondisi 2

```
margin of error
math.sqrt((p_hat * (1 - p_hat)) / n)

batas interval
p_hat - margin
p_hat + margin

hasil
Interval konfidensi      : ({batas_bawah:.4f}, {batas_atas:.4f})
Kesimpulan: {batas_bawah:.4f} < p < {batas_atas:.4f}"")

if __name__ == '__main__':
    interval_konfidensi(1.5, 0.1, 50)

*** Error: Proporsi harus antara 0 dan 1.
```

Keadaan 3

```
def interval_konfidensi(p_hat, alpha, n):
    print("Error: Nilai alpha harus 0.10 (10%) atau 0.05 (5%).")
    return

# Menghitung margin of error
margin = z * math.sqrt((p_hat * (1 - p_hat)) / n)

# Menghitung batas interval
batas_bawah = p_hat - margin
batas_atas = p_hat + margin

# Menampilkan hasil
print(f"Interval konfidensi      : ({batas_bawah:.4f}, {batas_atas:.4f})")
print(f"Kesimpulan: {batas_bawah:.4f} < p < {batas_atas:.4f}")

# Memanggil fungsi
interval_konfidensi(0.6, 0.6, 30)

*** Error: Nilai alpha harus 0.10 (10%) atau 0.05 (5%).
```

6.2.2 R

1. Program 1
 - a) Keadaan 1


```

> #PROGRAM 1
> #Keadaan 1
> # Program Menghitung Jumlah Kata dan Kalimat
> # Masukkan teks
> teks <- "Kehidupan anak statistika tidak jauh dari data, angka, dan berbagai tugas analisis. Setiap hari mereka belajar mengolah data untuk menemukan informasi yang dapat digunakan dalam pengambilan keputusan. Meskipun sering berhadapan dengan perhitungan yang rumit, anak statistika tetap menikmati proses belajar karena ilmu yang dipelajari sangat bermanfaat di berbagai bidang."
>
> # Hitung jumlah kata
> # Pisahkan berdasarkan satu atau lebih spasi
> kata <- (strsplit(trimws(teks), "\\s+")) [[1]]
> jumlah_kata<-length(kata) #Menghitung jumlah elemen dalam vector kata
>
> # Hitung jumlah kalimat
> # Pisahkan berdasarkan tanda titik
> kalimat <- strsplit(teks, "\\.") [[1]] #Asumsi tanda titik hanya untuk mengakhiri kalimat
> jumlah_kalimat<-length(kalimat) #Menghitung jumlah elemen dalam vector kalimat
>
> # Tampilkan hasil
> cat(sprintf("Jumlah kalimat : %d kalimat\n", jumlah_kalimat))
Jumlah kalimat : 3 kalimat
> cat(sprintf("Jumlah kata      : %d kata\n", jumlah_kata))
Jumlah kata      : 49 kata
> cat(sprintf("Teks tersebut memuat %d kalimat dan %d kata.\n",
+             jumlah_kalimat, jumlah_kata))
Teks tersebut memuat 3 kalimat dan 49 kata.
> |

```

b) Keadaan 2

```

> teks <- "olahraga merupakan aktivitas fisik yang sangat penting bagi kesehatan tubuh manusia. Dengan berolahraga secara teratur tubuh kita akan menjadi lebih bugar dan kuat. Banyak jenis olahraga yang bisa dipilih sesuai dengan minat dan kemampuan masing masing. Sepak bola basket renang dan lari adalah beberapa olahraga yang populer di Indonesia. Olahraga juga terbukti dapat mengurangi stres dan meningkatkan kesehatan mental seseorang. Pemerintah terus mendorong masyarakat untuk aktif berolahraga melalui berbagai program dan fasilitas umum. Mulailah berolahraga dari sekarang karena lebih baik mencegah daripada mengobati penyakit."
>
> # Hitung jumlah kata
> # Pisahkan berdasarkan satu atau lebih spasi
> kata <- (strsplit(trimws(teks), "\\s+")) [[1]]
> jumlah_kata<-length(kata) #Menghitung jumlah elemen dalam vector kata
>
> # Hitung jumlah kalimat
> # Pisahkan berdasarkan tanda titik
> kalimat <- strsplit(teks, "\\.") [[1]] #Asumsi tanda titik hanya untuk mengakhiri kalimat
> jumlah_kalimat<-length(kalimat) #Menghitung jumlah elemen dalam vector kalimat
>
> # Tampilkan hasil
> cat(sprintf("Jumlah kalimat : %d kalimat\n", jumlah_kalimat))
Jumlah kalimat : 7 kalimat
> cat(sprintf("Jumlah kata      : %d kata\n", jumlah_kata))
Jumlah kata      : 84 kata
> cat(sprintf("Teks tersebut memuat %d kalimat dan %d kata.\n",
+             jumlah_kalimat, jumlah_kata))
Teks tersebut memuat 7 kalimat dan 84 kata.
>

```

c) Keadaan 3

```

> # Program Menghitung Jumlah Kata dan Kalimat
> # Masukkan teks
> teks <- "Indonesia memiliki kekayaan kuliner tradisional yang sangat beragam dan lezat. . Setiap daerah di Indonesia memiliki makanan khas yang mencerminkan budaya setempat. Rendang dari Sumatera Barat diakui sebagai salah satu makanan terlezat di dunia. Soto gudeg nasi goreng dan gado gado juga merupakan makanan yang sangat terkenal. Makanan tradisional Indonesia menggunakan rempah rempah alami yang kaya manfaat bagi kesehatan. Kita harus melestarikan kuliner tradisional agar tidak punah tergerus oleh makanan modern. Memperkenalkan makanan tradisional kepada generasi muda adalah tanggung jawab kita bersama. Dengan mencintai kuliner lokal kita turut menjaga warisan budaya bangsa Indonesia."
>
> # Hitung jumlah kata
> # Pisahkan berdasarkan satu atau lebih spasi
> kata <- (strsplit(trimws(teks), "\\s+")) [[1]]
> jumlah_kata<-length(kata) #Menghitung jumlah elemen dalam vector kata
>
> # Hitung jumlah kalimat
> # Pisahkan berdasarkan tanda titik
> kalimat <- strsplit(teks, "\\.") [[1]] #Asumsi tanda titik hanya untuk mengakhiri kalimat
> jumlah_kalimat<-length(kalimat) #Menghitung jumlah elemen dalam vector kalimat
>
> # Tampilkan hasil
> cat(sprintf("Jumlah kalimat : %d kalimat\n", jumlah_kalimat))
Jumlah kalimat : 9 kalimat
> cat(sprintf("Jumlah kata      : %d kata\n", jumlah_kata))
Jumlah kata      : 93 kata
> cat(sprintf("Teks tersebut memuat %d kalimat dan %d kata.\n",
+             jumlah_kalimat, jumlah_kata))
Teks tersebut memuat 9 kalimat dan 93 kata.
> |

```

2. Program 2

```

> #PROGRAM 2
> K1 <- "Saya tak 'kan menyerah."
> K2 <- "Ia berkata, \"Aku menyayangimu.\"\" #Mengandung tanda kutip ganda (") di dalam string
> # Dalam R, tambahkan \" untuk menyisipkan tanda kutip ganda di dalam string
> K3 <- "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
> # K4 tidak ada karakter khusus yang perlu di-escape, ditulis langsung
> K4 <- "Surat keputusan itu bernomor 62/UN.34/19/2023."
> #Output
> cat("K1:", K1, "\n")
K1: Saya tak 'kan menyerah.
> cat("K2:", K2, "\n")
K2: Ia berkata, "Aku menyayangimu."
> cat("K3:", K3, "\n")
K3: "Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!"
> cat("K4:", K4, "\n")
K4: Surat keputusan itu bernomor 62/UN.34/19/2023.
> |

```

3. Program 3

a) Keadaan 1

```

> #Projek 3
> #Menghitung akar bilangan
> #Keadaan 1
> #Input persamaan
> a<-1
> b<-4
> c<-4
> #Input rumus D
> D<-b^2-4*a*c
> print(D)
[1] 0
> #Logika if-else
> if (D>0){
+   x1<-(-b+sqrt(D))/(2*a)
+   x2<-(-b-sqrt(D))/(2*a)
+   cat(sprintf("x1 = %.3f\n", x1))
+   cat(sprintf("x2 = %.3f\n", x2))
+ } else if (D==0){
+   x <- -b/(2*a)
+   cat(sprintf("x1 = x2 = %.3f\n", x))
+ } else {
+   cat("Persamaan hanya memiliki akar imajiner\n")
+ }
x1 = x2 = -2.000

```

b) Keadaan 2

```

> #Keadaan 2
> #Input persamaan
> a<-2
> b<-3
> c<-6
> #Input rumus D
> D<-b^2-4*a*c
> print(D)
[1] -39
> #Logika if-else
> if (D>0){
+   x1<-(-b+sqrt(D))/(2*a)
+   x2<-(-b-sqrt(D))/(2*a)
+   cat(sprintf("x1 = %.3f\n", x1))
+   cat(sprintf("x2 = %.3f\n", x2))
+ } else if (D==0){
+   x <- -b/(2*a)
+   cat(sprintf("x1 = x2 = %.3f\n", x))
+ } else {
+   cat("Persamaan hanya memiliki akar imajiner\n")
+ }
Persamaan hanya memiliki akar imajiner

```

c) Keadaan 3

```

~
> #Keadaan 3
> #Input persamaan
> a<-1
> b<- -5
> c<-6
> #Input rumus D
> D<-b^2-4*a*c
> print(D)
[1] 1
> #Logika if-else
> if (D>0){
+   x1<-(-b+sqrt(D))/(2*a)
+   x2<-(-b-sqrt(D))/(2*a)
+   cat(sprintf("x1 = %.3f\n", x1))
+   cat(sprintf("x2 = %.3f\n", x2))
+ } else if (D==0){
+   x <- -b/(2*a)
+   cat(sprintf("x1 = x2 = %.3f\n", x))
+ } else {
+   cat("Persamaan hanya memiliki akar imajiner\n")
+ }
x1 = 3.000
x2 = 2.000

```

4. Program 4

a) Keadaan 1

```

+   nama_bulan <- "Maret"
+ } else if (bulan == 4) {
+   nama_bulan <- "April"
+ } else if (bulan == 5) {
+   nama_bulan <- "Mei"
+ } else if (bulan == 6) {
+   nama_bulan <- "Juni"
+ } else if (bulan == 7) {
+   nama_bulan <- "Juli"
+ } else if (bulan == 8) {
+   nama_bulan <- "Agustus"
+ } else if (bulan == 9) {
+   nama_bulan <- "September"
+ } else if (bulan == 10) {
+   nama_bulan <- "Oktober"
+ } else if (bulan == 11) {
+   nama_bulan <- "November"
+ } else if (bulan == 12) {
+   nama_bulan <- "Desember"
+ }
+   cat("Tanggal lahir:", tanggal, nama_bulan, tahun, "\n")
+ }
+ }
Tanggal lahir: 21 April 1999
~

```

b) Keadaan 2

```
+ } else if (bulan == 4) {
+   nama_bulan <- "April"
+ } else if (bulan == 5) {
+   nama_bulan <- "Mei"
+ } else if (bulan == 6) {
+   nama_bulan <- "Juni"
+ } else if (bulan == 7) {
+   nama_bulan <- "Juli"
+ } else if (bulan == 8) {
+   nama_bulan <- "Agustus"
+ } else if (bulan == 9) {
+   nama_bulan <- "September"
+ } else if (bulan == 10) {
+   nama_bulan <- "Oktober"
+ } else if (bulan == 11) {
+   nama_bulan <- "November"
+ } else if (bulan == 12) {
+   nama_bulan <- "Desember"
+ }
+ cat("Tanggal lahir:", tanggal, nama_bulan, tahun, "\n")
+ }
+ }
Tanggal lahir: 16 september 1996
```

c) Keadaan 3

```
+ } else if (bulan == 4) {
+   nama_bulan <- "April"
+ } else if (bulan == 5) {
+   nama_bulan <- "Mei"
+ } else if (bulan == 6) {
+   nama_bulan <- "Juni"
+ } else if (bulan == 7) {
+   nama_bulan <- "Juli"
+ } else if (bulan == 8) {
+   nama_bulan <- "Agustus"
+ } else if (bulan == 9) {
+   nama_bulan <- "September"
+ } else if (bulan == 10) {
+   nama_bulan <- "Oktober"
+ } else if (bulan == 11) {
+   nama_bulan <- "November"
+ } else if (bulan == 12) {
+   nama_bulan <- "Desember"
+ }
+ cat("Tanggal lahir:", tanggal, nama_bulan, tahun, "\n")
+ }
+ }
Bulan tidak valid!
```

5. Program 5

a) Keadaan 1

```
> U<-c(x1,x2,x3)
> #definisi jarak U ke A, B, C
> dA<-jarak(U,A)
> dB<-jarak(U,B)
> dC<-jarak(U,C)
> #menentukan cluster
> if(dA<dB && dA<dC){
+   cluster<-"cluster A"
+ } else if (dB<dA && dB<dC){
+   cluster<-"cluster B"
+ } else if (dC<dA && dC<dB){
+   cluster<-"cluster C"
+ } else if (dA==dB && dA<dC){
+   cluster<-"Perbatasan cluster A dan B"
+ } else if (dA==dC && dA<dB){
+   cluster<-"Perbatasan cluster A dan C"
+ } else if (dB==dC && dB<dA){
+   cluster<-"Perbatasan cluster B dan C"
+ } else {
+   cluster<-"Perbatasan cluster A, B, dan C"
+ }
> #Output
> cat("Titik termasuk ", cluster, "\n")
Titik termasuk cluster B
```

b) Keadaan 2

```
> #input pusat dari tiga cluster
> A<-c(2,1,3)
> B<-c(1,-4,6)
> C<-c(-2,3,-2)
> #input U
> x1<-4
> x2<-2
> x3<-3
>
> U<-c(x1,x2,x3)
> #definisi jarak U ke A, B, C
> dA<-jarak(U,A)
> dB<-jarak(U,B)
> dC<-jarak(U,C)
> #menentukan cluster
> if(dA<dB && dA<dC){
+   cluster<-"cluster A"
+ } else if (dB<dA && dB<dC){
+   cluster<-"cluster B"
+ } else if (dC<dA && dC<dB){
+   cluster<-"cluster C"
+ } else if (dA==dB && dA<dC){
+   cluster<-"Perbatasan cluster A dan B"
+ } else if (dA==dC && dA<dB){
+   cluster<-"Perbatasan cluster A dan C"
+ } else if (dB==dC && dB<dA){
+   cluster<-"Perbatasan cluster B dan C"
+ } else {
+   cluster<-"Perbatasan cluster A, B, dan C"
+ }
> #Output
> cat("Titik termasuk ", cluster, "\n")
Titik termasuk cluster A
>
```

c) Keadaan 3

```
> #input U
> x1<-1.5
> x2<-1.5
> x3<-4.5
>
> U<-c(x1,x2,x3)
> #definisi jarak U ke A, B, C
> dA<-jarak(U,A)
> dB<-jarak(U,B)
> dC<-jarak(U,C)
> #menentukan cluster
> if(dA<dB && dA<dC){
+   cluster<-"cluster A"
+ } else if (dB<dA && dB<dC){
+   cluster<-"cluster B"
+ } else if (dC<dA && dC<dB){
+   cluster<-"cluster C"
+ } else if (dA==dB && dA<dC){
+   cluster<-"Perbatasan cluster A dan B"
+ } else if (dA==dC && dA<dB){
+   cluster<-"Perbatasan cluster A dan C"
+ } else if (dB==dC && dB<dA){
+   cluster<-"Perbatasan cluster B dan C"
+ } else {
+   cluster<-"Perbatasan cluster A, B, dan C"
+ }
> #Output
> cat("Titik termasuk ", cluster, "\n")
Titik termasuk Perbatasan cluster A dan B
>
```

6. Program 6

a) Keadaan 1

```
> #keadaan 1
> interval_proporsi<-function(phat, n, alpha){
+   if(phat<0||phat>1){
+     cat("error: proporsi harus antara 0 dan 1\n")
+     return()
+   }
+   if(alpha==0.10){
+     z<-1.645
+   }else if(alpha==0.05){
+     z<-1.96
+   }else {
+     cat("Error: nilai alpha harus 0.10 (10%) atau 0.05(5%) \n")
+     return()
+   }
+   margin<-z*sqrt((phat*(1-phat))/n)
+   bawah<-phat-margin
+   atas <- phat + margin
+   cat("Interval Konfidensi:\n")
+   cat("(", bawah, ", ", atas, ")\n")
+ }
> phat<-0.3
> n <-15
> alpha <- 0.05
> interval_proporsi(phat,n,alpha)
Interval Konfidensi:
( 0.06808967 , 0.5319103 )
```

b) Keadaan 2

```
> interval_proporsi<-function(phat, n, alpha){
+   if(phat<0||phat>1){
+     cat("error: proporsi harus antara 0 dan 1\n")
+     return()
+   }
+   if(alpha==0.10){
+     z<-1.645
+   }else if(alpha==0.05){
+     z<-1.96
+   }else {
+     cat("Error: nilai alpha harus 0.10 (10%) atau 0.05(5%) \n")
+     return()
+   }
+   margin<-z*sqrt((phat*(1-phat))/n)
+   bawah<-phat-margin
+   atas <- phat + margin
+   cat("Interval Konfidensi:\n")
+   cat("(", bawah, ", ", atas, ")\n")
+ }
> phat<-0.6
> n <-30
> alpha <- 0.6
> interval_proporsi(phat,n,alpha)
Error: Nilai alpha harus 0.10 (10%) atau 0.05(5%)
NULL
```

c) Keadaan 3

```

> #keadaan 2
> interval_proporsi<-function(phat, n, alpha){
+   if(phat<0||phat>1){
+     cat("error: proporsi harus antara 0 dan 1\n")
+     return()
+   }
+   if(alpha==0.10){
+     z<-1.645
+   }else if(alpha==0.05){
+     z<-1.96
+   }else {
+     cat("Error: Nilai alpha harus 0.10 (10%) atau 0.05(5%) \n")
+     return()
+   }
+   margin<-z*sqrt((phat*(1-phat))/n)
+   bawah<-phat-margin
+   atas <- phat + margin
+   cat("Interval Konfidensi:\n")
+   cat("(", bawah, ", ", atas, ")\n")
+ }
> phat<-1.5
> n <-50
> alpha <- 0.10
> interval_proporsi(phat,n,alpha)

```

```

error: proporsi harus antara 0 dan 1
NULL

```

6.3 Interpretasi Output

6.3.1 Program 1

Hasil pengujian menunjukkan bahwa program mampu menghitung jumlah kata dan jumlah kalimat dari teks yang diberikan. Jumlah kata diperoleh dengan memisahkan teks berdasarkan spasi menggunakan fungsi `strsplit()`, sedangkan jumlah kalimat dihitung berdasarkan banyaknya tanda titik (.) yang berfungsi sebagai penanda akhir kalimat. Pada setiap pengujian, program berhasil menampilkan jumlah kata dan jumlah kalimat sesuai dengan isi teks yang dimasukkan. Hal ini menunjukkan bahwa algoritma yang digunakan telah berjalan dengan baik selama teks memenuhi asumsi bahwa tanda titik hanya digunakan sebagai penutup kalimat.

6.3.2 Program 2

Hasil pengujian menunjukkan bahwa seluruh objek string, yaitu K1, K2, K3, dan K4, berhasil ditampilkan sesuai dengan format yang diminta pada soal. Program mampu menampilkan karakter khusus seperti tanda petik tunggal (') seperti di K1, pada K2 program berhasil menampilkan tanda petik ganda di dalam string dengan menggunakan karakter escape (\ ") sehingga output yang dihasilkan sesuai dengan format yang diminta pada soal. Pada K3, program mampu menampilkan kombinasi tanda petik ganda dan tanda petik tunggal dalam satu string tanpa mengubah isi teks, hal ini menunjukkan penggunaan karakter *escape* telah dilakukan dengan benar. Sedangkan pada K4 program berhasil menampilkan string yang berisi kombinasi huruf, angka, garis miring (/), dan tanda titik (.) tanpa kesalahan. Seluruh isi string sesuai dengan yang didefinisikan pada program.

6.3.3 Program 3

Pada keadaan 1, program berhasil menghitung dua akar real kembar karena nilai diskriminan $D = 0$, maka program terbukti mampu menangani kasus akar kembar dengan

benar. Selanjutnya pada keadaan 2, didapati nilai diskriminan $D < 0$, maka sesuai program output menunjukkan persamaan tidak memiliki akar real. Pada keadaan 3, diskriminan bernilai $D > 0$ persamaan memiliki dua akar real berbeda, dan output menyatakan dua akar real berbeda yaitu $x_1 = 3$ dan $x_2 = 2$.

6.3.4 Program 4

Program diambil empat digit pertama sebagai tahun lahir, dua digit selanjutnya sebagai bulan, dan dua digit selanjutnya sebagai tanggal lahir. Nama bulan ditampilkan menggunakan percabangan sehingga menghasilkan tanggal lahir yang benar untuk keadaan satu dimana output menampilkan 21 April 1999. Hal serupa terjadi pada keadaan dua dimana input normal sehingga menghasilkan output 16 September 1996. Pada keadaan tiga, input bulan tidak masuk kondisi if sehingga output berupa 15 Bulan tidak Valid 1995.

6.3.5 Program 5

Program ini digunakan untuk mengklasifikasikan titik tersebut termasuk cluster A, B, C, atau perbatasan sesuai dengan jarak terkecil dari titik pusat yang telah ditentukan hingga vektor U. Pada keadaan 1, perhitungan menunjukkan jarak U-B merupakan jarak terkecil sehingga titik termasuk cluster B. Program yang serupa berjalan pada keadaan dua dimana jarak U-A terkecil sehingga titik termasuk cluster A. Pada keadaan 3 ditemukan bahwa jarak U-A sama dengan jarak U-B sehingga titik termasuk perbatasan antara cluster A dan B.

6.3.6 Program 6

Pada keadaan satu program berhasil menghitung interval konfidensi menggunakan proporsi sampel, ukuran sampel, dan nilai $\alpha = 0,05$. Nilai z yang digunakan adalah 1,96 sehingga diperoleh batas bawah dan batas atas interval konfidensi sesuai dengan rumus yang diberikan yaitu 0.0681 dan 0.5319. Selanjutnya pada keadaan dua program tidak berhasil menghitung interval karena $p > 1$ sehingga output berupa error karena proporsi harus antara 0 dan 1. Terakhir pada keadaan 3 program tidak berhasil menghitung interval karena $\alpha \neq 0.05$ dan $\alpha \neq 0.10$ sehingga output menampilkan Error: Nilai alpha harus 0.10 (10%) atau 0.05 (5%).

BAB VII PENUTUP

7.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa seluruh program pada project Algoritma dan Pemrograman berhasil dikembangkan sesuai dengan tujuan yang telah ditetapkan. Keenam program, diantaranya program analisis teks, pembuatan objek string, pencarian akar persamaan kuadrat, penentuan tanggal lahir berdasarkan NIP ASN, klasifikasi cluster tiga dimensi, serta perhitungan interval konfidensi proporsi, telah berhasil diimplementasikan menggunakan bahasa pemrograman R dan Python dengan logika yang setara.

Hasil pengujian menunjukkan bahwa setiap program mampu menghasilkan keluaran yang sesuai dengan spesifikasi soal pada kondisi masukan normal. Selain itu, beberapa program jugatelah dilengkapi dengan validasi input, seperti pengecekan nilai proporsi pada program interval konfidensi serta pengecekan bulan pada program pengolahan NIP ASN, sehingga program dapat memberikan pesan kesalahan apabila data yang dimasukkan tidak sesuai dengan ketentuan. Penggunaan fungsi pada program klasifikasi cluster dan interval konfidensi juga menunjukkan penerapan konsep modularisasi sehingga kode menjadi lebih terstruktur, mudah dipahami, dan mudah digunakan kembali.

Melalui proyek ini, diperoleh pemahaman yang lebih baik mengenai penerapan konsep dasar algoritma, penggunaan percabangan, fungsi, operasi string, operasi matematika, serta pengolahan data masukan dalam pembuatan program. Selain itu, proyek ini juga menunjukkan pentingnya proses pengujian untuk memastikan setiap program dapat berjalan sesuai dengan logika yang dirancang dan menghasilkan keluaran yang tepat.

7.2 Kendala yang Dihadapi

Selama proses pengerjaan project, terdapat beberapa kendala yang dihadapi oleh kelompok. Salah satunya adalah proses menerjemahkan algoritma yang sama ke dalam dua bahasa pemrograman yang memiliki sintaks berbeda, yaitu R dan Python. Perbedaan struktur penulisan, fungsi bawaan, serta cara menerima input dari pengguna menyebabkan beberapa penyesuaian perlu dilakukan agar kedua program tetap menghasilkan keluaran yang setara.

Kendala lainnya adalah proses penanganan kondisi khusus pada beberapa program, seperti menentukan akar persamaan kuadrat ketika diskriminan bernilai negatif, mengolah string yang mengandung karakter khusus, menentukan nama bulan dari NIP ASN, serta melakukan validasi terhadap nilai proporsi agar berada pada rentang yang benar. Selain itu, proses pembuatan flowchart, pseudocode, dokumentasi hasil pengujian, dan penjelasan setiap baris kode membutuhkan ketelitian serta koordinasi yang baik antar anggota kelompok agar seluruh komponen laporan tetap konsisten dengan implementasi program.

7.3 Saran Pengembangan Program

Program yang telah dibuat masih dapat dikembangkan lebih lanjut agar memiliki fungsionalitas yang lebih baik. Pada program analisis teks, perhitungan jumlah kalimat dapat diperbaiki dengan

mengenali berbagai tanda baca seperti tanda tanya dan tanda seru serta mengabaikan tanda titik yang tidak berfungsi sebagai akhir kalimat. Program pembuatan string dapat dikembangkan agar menerima masukan langsung dari pengguna sehingga lebih fleksibel.

Pada program akar persamaan kuadrat, pengembangan dapat dilakukan dengan menampilkan akar bilangan kompleks ketika diskriminan bernilai negatif, bukan hanya memberikan informasi bahwa akar bersifat imajiner. Program pengolahan NIP ASN juga dapat dilengkapi dengan validasi panjang NIP serta pemeriksaan format tanggal sehingga kesalahan input dapat dideteksi lebih awal.

Untuk program klasifikasi cluster, pengembangan dapat dilakukan dengan memvisualisasikan posisi titik dalam ruang tiga dimensi. Sementara itu, program interval konfidensi proporsi dapat dikembangkan dengan mendukung lebih banyak tingkat kepercayaan, melakukan validasi terhadap ukuran sampel, serta menyajikan hasil perhitungan dalam bentuk visualisasi grafik agar interpretasi hasil menjadi lebih mudah dipahami.

Secara keseluruhan, pengembangan tersebut diharapkan dapat meningkatkan keandalan, fleksibilitas, dan kemudahan penggunaan program sehingga mampu diterapkan pada permasalahan yang lebih kompleks di masa mendatang.

LAMPIRAN

Pembagian Tugas Kelompok

Nama Mahasiswa	Bagian Tugas Kelompok
Sekar Sandy Setiawan	Membuat source code R untuk program 3-6. Menyusun ppt presentasi untuk program 5. Menyusun laporan bab 3, bab 6, dan bab 7.
Keyko Anastasia Molshac	Membuat pseudocode untuk program 1-6. Menyusun ppt presentasi untuk program 4. Menyusun laporan bab 1, bab 2, dan bab 4.
Fatmawaty Dwi Yunianti	Membuat source code R untuk program 1 dan 2. Membuat source code Python untuk program 5 dan 6. Menyusun ppt presentasi untuk program 6. Menyusun laporan bab 2. Membuat demo pada R.
Julius Raymond	Membuat source code Python untuk program 1-4. Menyusun ppt presentasi untuk program 1 dan 2. Membuat demo pada Python.
Ilham Luqmanulhakim	Membuat flowchart untuk program 1-6. Menyusun ppt untuk program 3 Menyusun laporan bab 3, dan bab 5

<https://github.com/Julius-Ray/Tugas-Projek-Alpro-A>

https://drive.google.com/drive/folders/1UmnN_la6JEF2YqitkbhDTuQ0IyCBb6DA

<https://claude.ai/share/a3321d22-1054-437b-a37d-d367c3fd0d2f>